

STAT 371 Group Project

Lei Yang, Catherine Chen, Jessica Huang

2026-04-01

Research Question and Data Description

This project investigates which automobile characteristics are associated with vehicle price from the UCI Automobile dataset. Specifically, we explore whether we can use a multiple linear regression model, combining continuous predictor variables such as horsepower, curb weight, urban fuel efficiency, standardized loss and vehicle length, with categorical predictor variables like brand, drive type, and body style, to explain differences in vehicle prices.

This research question is interesting as car prices reflect the combined effects of multiple factors, including engineering, design, performance, and market positioning. Understanding the features most closely linked to price helps explain how vehicle attributes influence market value.

The dataset used in this analysis is the Automobile dataset obtained from the UCI Machine Learning Repository. After data cleaning, the dataset consists of **205 observations** and **26 variables**, with **price** as the response variable.

The explanatory variables include both continuous and categorical predictors. Continuous variables capture vehicle size, engine characteristics, and fuel efficiency (e.g., engine_size, horsepower, city_mpg), while categorical variables describe attributes such as make, body_style, drive_wheels, and fuel_type.

Missing values were handled during data preparation, and categorical variables were converted into factors where appropriate.

Literature Review

The UCI Automobile dataset has been used and referenced in a range of prior work, but it is often used as a supporting example rather than as the main substantive focus. The dataset frequently serves as a benchmark or illustrative application for a broader methodological contribution. The UCI repository describes the dataset as a regression dataset with 205 observations, 25 features, and missing values, which makes it a convenient choice for demonstrating statistical and machine-learning procedures.

For instance, Cai et al. (2019) used the UCI dataset as the final example in their study of Bayesian inverse regression for supervised dimension reduction in small datasets. The categorical attributes were removed and the observations with missing values were deleted by the authors, leaving 159 observations and 16 numeric attributes. price was then used as the response variable in a reduced-dimension regression framework. However, the main purpose of the paper was to assess the performance of the proposed dimension-reduction method in a small-sample setting, instead of investigating automobile pricing itself.

A similar case study in research on integrating data mining and data warehousing was conducted by Usman and Pears (2010). Their emphasis was on broader data-processing tasks, such as handling mixed nominal and numeric data, clustering, data transformation, and warehouse-schema design, not on interpretable regression modeling of automobile price.

In addition to academic papers, public GitHub projects have also used the UCI Automobile dataset for vehicle price prediction. One representative project approaches the problem through a linear-regression-based prediction workflow, including missing value treatment, assumption checking, multicollinearity assessment, feature engineering, feature selection, and regularization. Even so, the main emphasis of that project remains predictive workflow and model performance rather than a course-style decision trail documenting each modeling stages and choices (Chakraborty & Samia, 2021).

Our Contribution

Our project distinguishes from these earlier uses in the following aspects:

1. We treat the UCI Automobile dataset itself as the main subject of analysis, instead of treating the dataset as a convenient example solely for another methodological objective.
2. Beginning with the initial raw data, we document the full regression process, covering data preparation, missing value assessment, exploratory data analysis, candidate predictor selection, model building, model check, interpretation, and final presentation delivery.
3. We preserve predictor-side missingness during initial cleaning rather than immediately deleting all incomplete observations, since our project emphasizes statistical decision-making transparency and diagnostic reasoning in addition to model fit.

Data Cleaning and Preparation

Column names are defined based on the Attribute Information section.

```
cols <- c(
  "symboling", "normalized_losses", "make", "fuel_type", "aspiration",
  "num_doors", "body_style", "drive_wheels", "engine_location",
  "wheel_base", "length", "width", "height", "curb_weight",
  "engine_type", "num_cylinders", "engine_size", "fuel_system",
  "bore", "stroke", "compression_ratio", "horsepower",
  "peak_rpm", "city_mpg", "highway_mpg", "price"
)
```

Column types follow UCI descriptions: numeric (continuous) and factor (categorical).

```
col_classes <- c(
  "factor", "numeric", "factor", "factor", "factor",
  "factor", "factor", "factor", "factor",
  "numeric", "numeric", "numeric", "numeric", "numeric",
  "factor", "factor", "numeric", "factor",
  "numeric", "numeric", "numeric", "numeric",
  "numeric", "numeric", "numeric", "numeric"
)
```

The Automobile dataset is loaded from UCI, with ? converted to NA and column names assigned.

```
data_url <- "https://archive.ics.uci.edu/ml/machine-learning-databases/autos/imports-85.data"

auto <- read.csv(
  file = data_url,
  header = FALSE,
  na.strings = "?",
  colClasses = col_classes
)

colnames(auto) <- cols
```

Calculate the number and percentage of missing values for each column.

```
na_count <- colSums(is.na(auto))
na_pct <- round(na_count / nrow(auto) * 100, 1)
na_summary <- data.frame(count = na_count, pct = na_pct)
na_summary[na_summary$count > 0, ] # Display only columns with missing values
```

	count	pct
normalized_losses	41	20
num_doors	2	1
bore	4	2
stroke	4	2
horsepower	2	1
peak_rpm	2	1
price	4	2

Observations with missing values in the response variable (price) are removed, and the remaining missing data are then examined.

```
auto_clean <- auto[!is.na(auto$price), ]
auto_clean[!complete.cases(auto), c("make", "normalized_losses",
                                     "bore", "stroke", "horsepower", "num_doors")]
```

	make	normalized_losses	bore	stroke	horsepower	num_doors
1	alfa-romero	NA	3.47	2.68	111	two
2	alfa-romero	NA	3.47	2.68	111	two
3	alfa-romero	NA	2.68	3.47	154	two
6	audi	NA	3.19	3.40	110	two
8	audi	NA	3.19	3.40	110	four
11	bmw	192	3.50	2.80	101	two
16	bmw	NA	3.62	3.39	182	four
17	bmw	NA	3.62	3.39	182	two
18	bmw	NA	3.62	3.39	182	four
19	chevrolet	121	2.91	3.03	48	two
29	dodge	110	3.34	3.46	88	four
47	isuzu	NA	3.43	3.23	90	two
48	jaguar	145	3.63	4.17	176	four

49	jaguar	NA	3.63	4.17	176	four
50	jaguar	NA	3.54	2.76	262	two
52	mazda	104	3.03	3.15	68	two
53	mazda	104	3.03	3.15	68	two
59	mazda	150	NA	NA	135	two
60	mazda	129	3.39	3.39	84	two
61	mazda	115	3.39	3.39	84	four
62	mazda	129	3.39	3.39	84	two
67	mazda	NA	3.43	3.64	72	four
70	mercedes-benz	93	3.58	3.64	123	two
75	mercedes-benz	NA	3.80	3.35	184	two
77	mitsubishi	161	2.97	3.23	68	two
78	mitsubishi	161	2.97	3.23	68	two
79	mitsubishi	161	2.97	3.23	68	two
86	mitsubishi	125	3.35	3.46	88	four
87	mitsubishi	125	3.35	3.46	88	four
88	mitsubishi	125	3.17	3.46	116	four
113	peugot	161	3.70	3.52	95	four
114	peugot	NA	3.46	2.19	95	four
117	peugot	161	3.70	3.52	95	four
118	peugot	161	3.61	3.21	142	four
128	porsche	NA	3.74	2.90	207	two
131	renault	NA	3.46	3.90	NA	four
132	renault	NA	3.46	3.90	NA	two
133	saab	150	3.54	3.07	110	two
134	saab	104	3.54	3.07	110	four
135	saab	150	2.54	2.07	110	two
136	saab	104	3.54	3.07	110	four
186	volkswagen	94	3.19	3.40	85	four
194	volkswagen	NA	3.19	3.40	88	four
196	volvo	74	3.78	3.15	114	four
197	volvo	103	3.78	3.15	114	four
198	volvo	74	3.78	3.15	114	four

Two observations with missing categorical values were removed due to their small proportion.

```
auto_clean <- auto_clean[!is.na(auto_clean$num_doors), ]
auto_clean[!complete.cases(auto), c("make", "normalized_losses",
                                     "bore", "stroke", "horsepower", "num_doors")]
```

	make	normalized_losses	bore	stroke	horsepower	num_doors
1	alfa-romero	NA	3.47	2.68	111	two
2	alfa-romero	NA	3.47	2.68	111	two
3	alfa-romero	NA	2.68	3.47	154	two
6	audi	NA	3.19	3.40	110	two
8	audi	NA	3.19	3.40	110	four
11	bmw	192	3.50	2.80	101	two
16	bmw	NA	3.62	3.39	182	four

17	bmw	NA	3.62	3.39	182	two
18	bmw	NA	3.62	3.39	182	four
19	chevrolet	121	2.91	3.03	48	two
30	dodge	145	3.60	3.90	145	two
48	jaguar	145	3.63	4.17	176	four
49	jaguar	NA	3.63	4.17	176	four
50	jaguar	NA	3.54	2.76	262	two
51	mazda	104	3.03	3.15	68	two
53	mazda	104	3.03	3.15	68	two
54	mazda	113	3.03	3.15	68	four
60	mazda	129	3.39	3.39	84	two
61	mazda	115	3.39	3.39	84	four
62	mazda	129	3.39	3.39	84	two
63	mazda	115	3.39	3.39	84	four
69	mercedes-benz	93	3.58	3.64	123	four
72	mercedes-benz	NA	3.46	3.10	155	four
77	mitsubishi	161	2.97	3.23	68	two
79	mitsubishi	161	2.97	3.23	68	two
80	mitsubishi	161	3.03	3.39	102	two
81	mitsubishi	153	3.17	3.46	116	two
88	mitsubishi	125	3.17	3.46	116	four
89	mitsubishi	137	3.17	3.46	116	four
90	nissan	128	3.15	3.29	69	two
115	peugot	NA	3.70	3.52	95	four
116	peugot	161	3.46	3.19	97	four
119	plymouth	119	2.97	3.23	68	two
120	plymouth	119	3.03	3.39	102	two
131	renault	NA	3.46	3.90	NA	four
133	saab	150	3.54	3.07	110	two
134	saab	104	3.54	3.07	110	four
135	saab	150	2.54	2.07	110	two
136	saab	104	3.54	3.07	110	four
137	saab	150	3.54	3.07	160	two
138	saab	104	3.54	3.07	160	four
188	volkswagen	94	3.01	3.40	68	four
196	volvo	74	3.78	3.15	114	four
198	volvo	74	3.78	3.15	114	four
199	volvo	103	3.62	3.15	162	four
200	volvo	74	3.62	3.15	162	four

```

na_count <- colSums(is.na(auto_clean))
na_pct <- round(na_count / nrow(auto_clean) * 100, 1)
na_summary <- data.frame(count = na_count, pct = na_pct)
na_summary[na_summary$count > 0, ] # Display only columns with missing values

```

	count	pct
normalized_losses	36	18.1
bore	4	2.0

```
stroke          4  2.0
horsepower      2  1.0
peak_rpm        2  1.0
```

Impute missing values in continuous variables using the median.

```
auto_clean$normalized_losses[is.na(auto_clean$normalized_losses)] <- median(auto_clean$normalized_losses,
  ↪ na.rm = TRUE)
auto_clean$bore[is.na(auto_clean$bore)] <- median(auto_clean$bore, na.rm = TRUE)
auto_clean$stroke[is.na(auto_clean$stroke)] <- median(auto_clean$stroke, na.rm = TRUE)
auto_clean$horsepower[is.na(auto_clean$horsepower)] <- median(auto_clean$horsepower, na.rm = TRUE)
auto_clean$peak_rpm[is.na(auto_clean$peak_rpm)] <- median(auto_clean$peak_rpm, na.rm = TRUE)

sum(is.na(auto_clean)) # Should be 0
```

```
[1] 0
```

```
nrow(auto_clean)
```

```
[1] 199
```

Exploratory Data Analysis

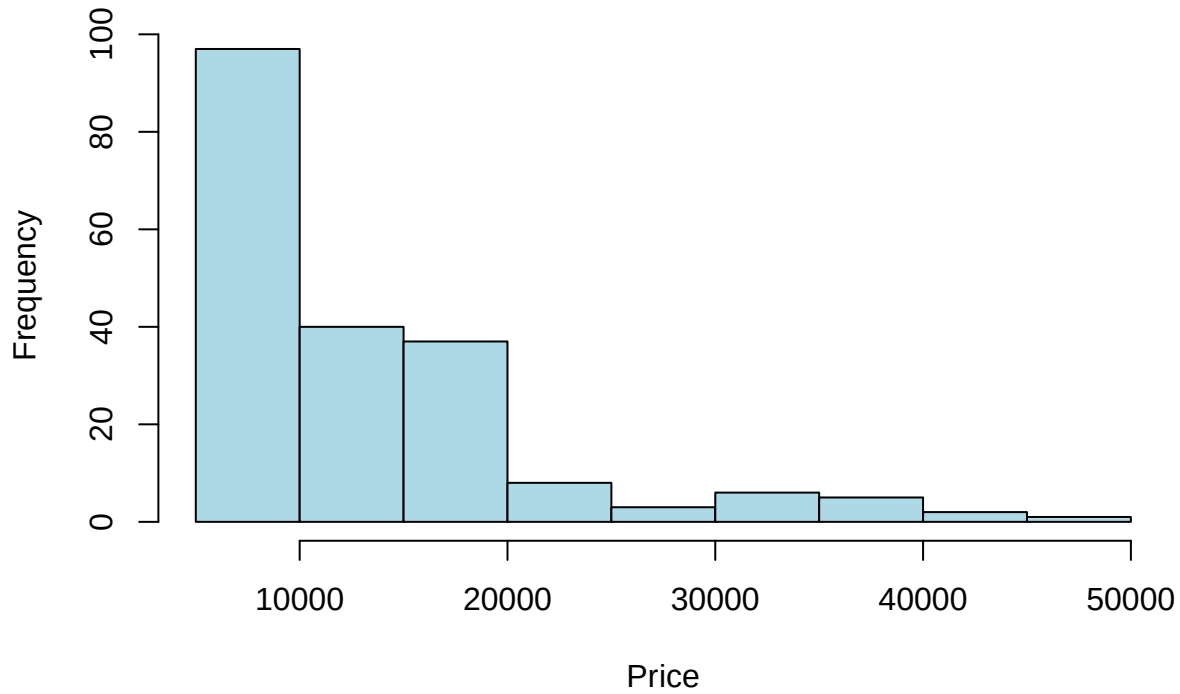
Response Variable Distribution

```
# Examine the distribution of the response variable (price)
summary(auto_clean$price)
```

```
Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
5118   7775   10295   13243   16502   45400
```

```
hist(auto_clean$price,
  main = "Distribution of Vehicle Price",
  xlab = "Price",
  col = "lightblue")
```

Distribution of Vehicle Price



Observations:

The distribution of price is right-skewed, with most vehicles clustered at lower prices and a few high-value observations forming a long tail.

Summary Statistics

Table 1: Summary of Categorical Variables

Variable	Category	Frequency
symboling	-1	22
symboling	-2	3
symboling	0	64
symboling	1	51
symboling	2	32
symboling	3	27
make	alfa-romero	3
make	audi	6
make	bmw	8
make	chevrolet	3
make	dodge	8
make	honda	13
make	isuzu	2
make	jaguar	3
make	mazda	16
make	mercedes-benz	8
make	mercury	1
make	mitsubishi	13

Variable	Category	Frequency
make	nissan	18
make	peugot	11
make	plymouth	7
make	porsche	4
make	renault	2
make	saab	6
make	subaru	12
make	toyota	32
make	volkswagen	12
make	volvo	11
fuel_type	diesel	19
fuel_type	gas	180
body_style	convertible	6
body_style	hardtop	8
body_style	hatchback	68
body_style	sedan	92
body_style	wagon	25

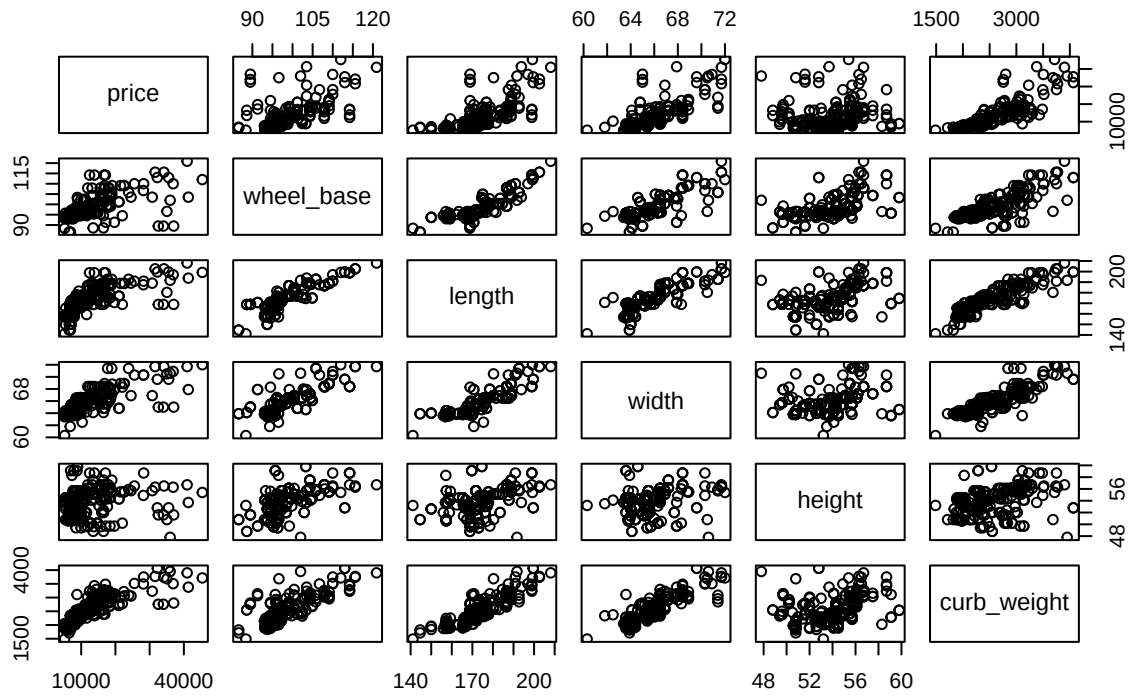
Table 2: Summary Statistics of Numeric Variables

Variable	Min	Q1	Median	Mean	Q3	Max	SD
normalized_losses	65.00	99.50	115.00	120.60	137.00	256.00	32.21
wheel_base	86.60	94.50	97.00	98.82	102.40	120.90	6.09
length	141.10	166.80	173.20	174.27	183.50	208.10	12.32
width	60.30	64.15	65.50	65.90	66.75	72.00	2.11
height	47.80	52.00	54.10	53.77	55.55	59.80	2.45
curb_weight	1488.00	2157.00	2414.00	2558.07	2930.50	4066.00	519.19
engine_size	61.00	97.50	120.00	127.05	143.00	326.00	41.70
bore	2.54	3.15	3.31	3.33	3.58	3.94	0.27
stroke	2.07	3.11	3.29	3.26	3.41	4.17	0.32
compression_ratio	7.00	8.60	9.00	10.11	9.40	23.00	3.92
horsepower	48.00	70.00	95.00	103.52	116.00	262.00	37.46
peak_rpm	4150.00	4800.00	5200.00	5118.84	5500.00	6600.00	478.67
city_mpg	13.00	19.00	24.00	25.13	30.00	49.00	6.41
highway_mpg	16.00	25.00	30.00	30.63	34.00	54.00	6.80
price	5118.00	7775.00	10295.00	13242.61	16501.50	45400.00	7978.34

Relationship: Price vs Continuous Variables

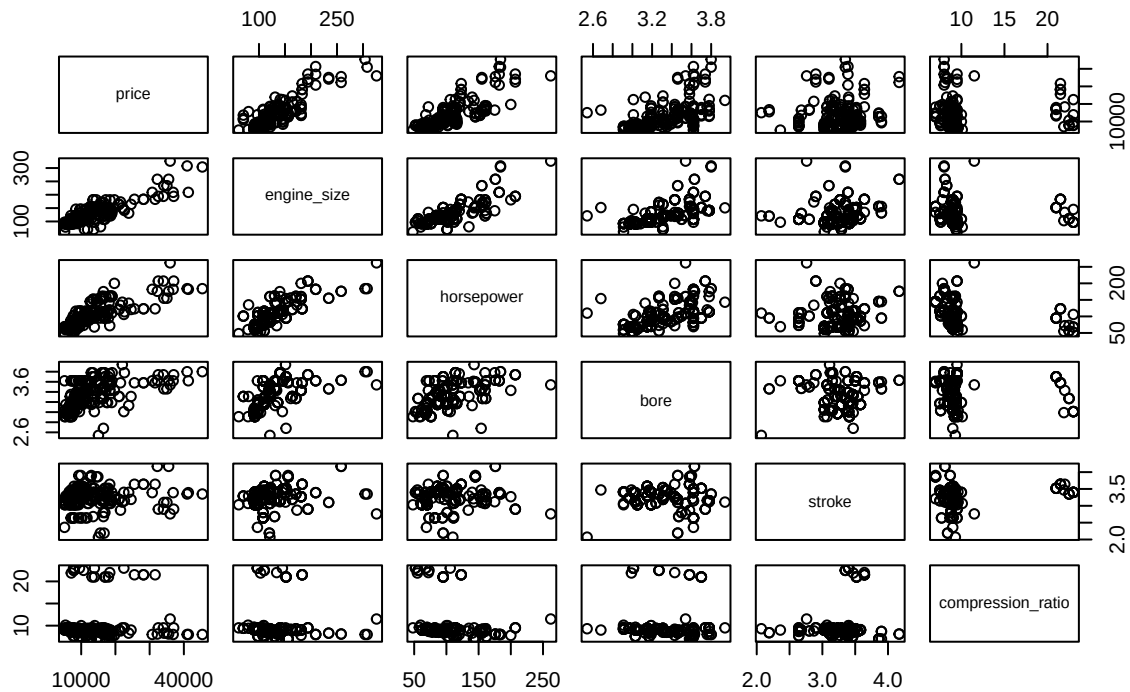
```
# Group 1: Vehicle size and weight
pairs(~ price + wheel_base + length + width + height + curb_weight,
      data = auto_clean,
      main = "Price vs Vehicle Dimensions")
```

Price vs Vehicle Dimensions

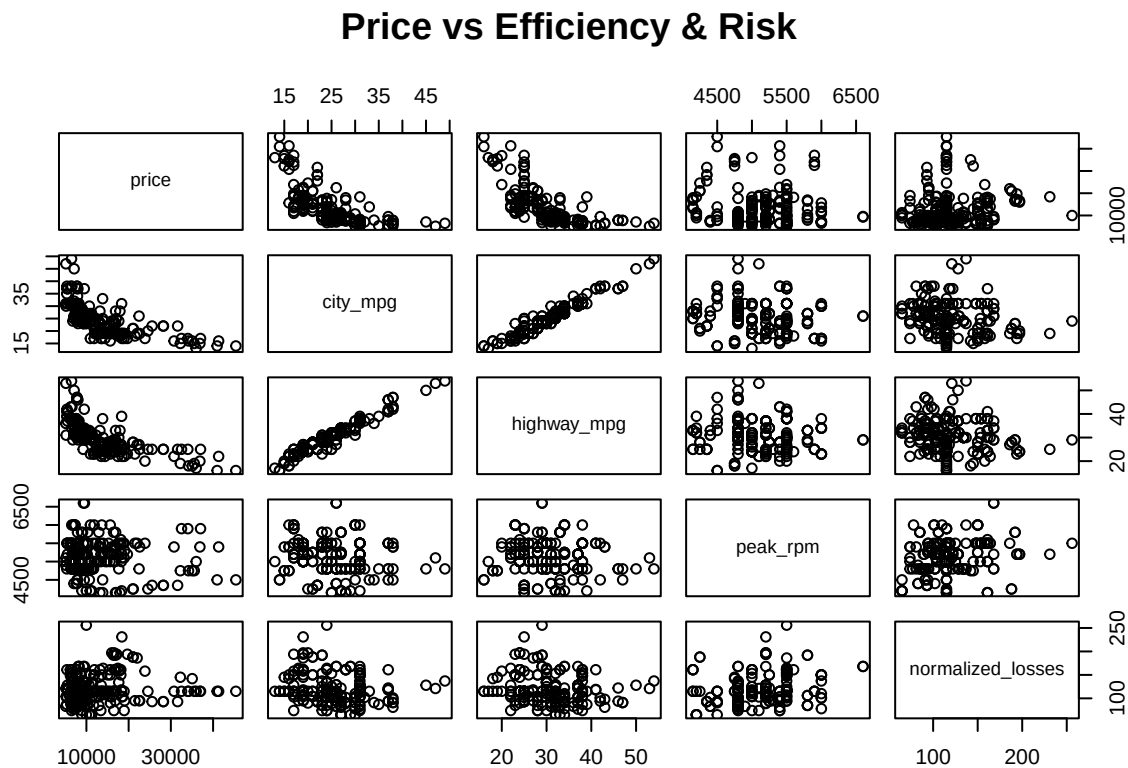


```
# Group 2: Engine and performance characteristics
pairs(~ price + engine_size + horsepower + bore + stroke + compression_ratio,
      data = auto_clean,
      main = "Price vs Engine Specifications")
```

Price vs Engine Specifications



```
# Group 3: Fuel efficiency and risk-related variables
pairs(~ price + city_mpg + highway_mpg + peak_rpm + normalized_losses,
      data = auto_clean,
      main = "Price vs Efficiency & Risk")
```

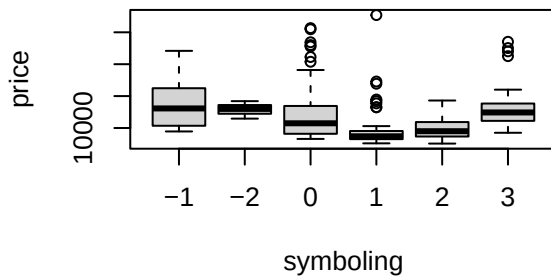


Price vs Categorical Variables

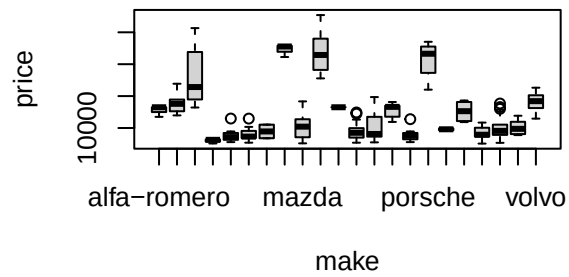
```
# Compare price distribution across categories
par(mfrow = c(2, 2))

boxplot(price ~ symboling, data = auto_clean, main = "Price by Risk Level")
boxplot(price ~ make, data = auto_clean, main = "Price by Manufacturer")
boxplot(price ~ aspiration, data = auto_clean, main = "Price by Aspiration Type")
boxplot(price ~ num_doors, data = auto_clean, main = "Price by Number of Doors")
```

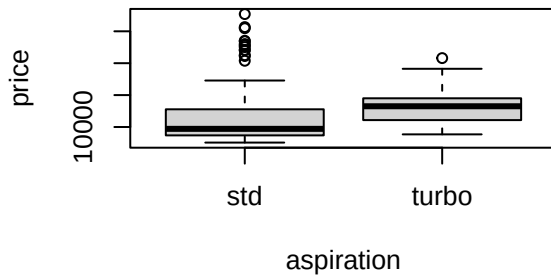
Price by Risk Level



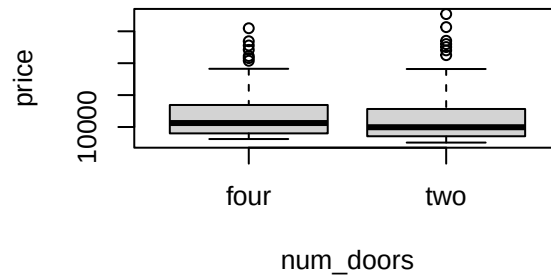
Price by Manufacturer



Price by Aspiration Type

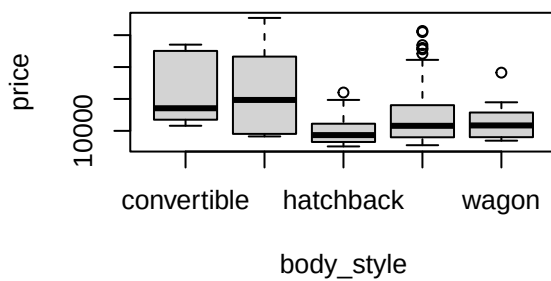


Price by Number of Doors

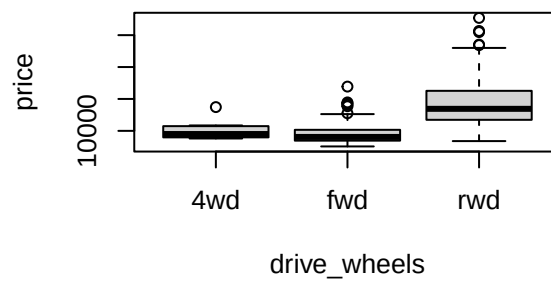


```
boxplot(price ~ body_style, data = auto_clean, main = "Price by Body Style")
boxplot(price ~ drive_wheels, data = auto_clean, main = "Price by Drive Type")
boxplot(price ~ fuel_type, data = auto_clean, main = "Price by Fuel Type")
boxplot(price ~ engine_location, data = auto_clean, main = "Price by Engine Location")
```

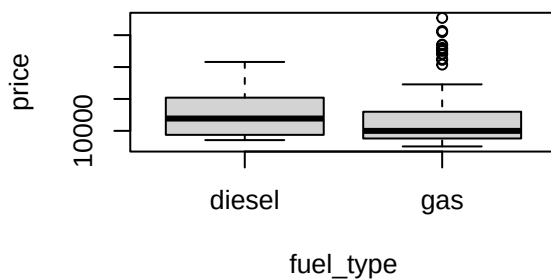
Price by Body Style



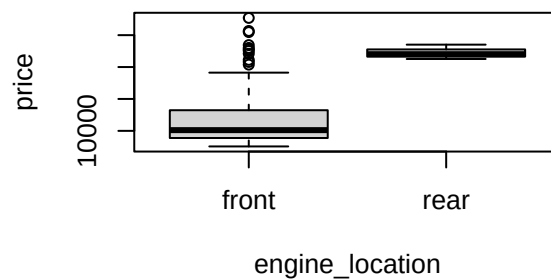
Price by Drive Type



Price by Fuel Type



Price by Engine Location



```

boxplot(price ~ engine_type, data = auto_clean, main = "Price by Engine Type")

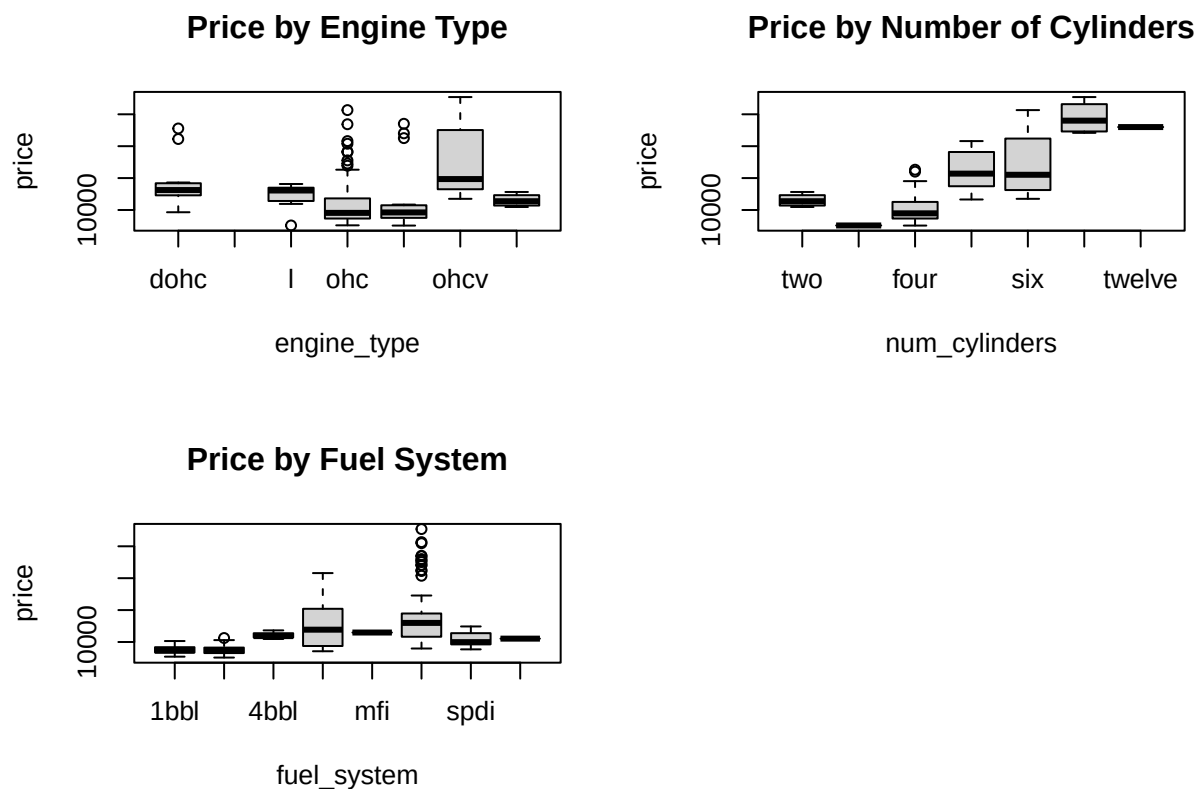
# Convert number of cylinders to ordered factor
auto_clean$num_cylinders <- factor(auto_clean$num_cylinders,
                                   levels = c("two", "three", "four", "five",
                                               "six", "eight", "twelve"))

boxplot(price ~ num_cylinders, data = auto_clean,
        main = "Price by Number of Cylinders")

boxplot(price ~ fuel_system, data = auto_clean,
        main = "Price by Fuel System")

par(mfrow = c(1, 1))

```



```

# Observations:

# Group 1: Vehicle size and weight
# The pair plot indicates that price is strongly related to size-related variables, especially
↳ curb_weight.
# Strong multicollinearity exists among wheel_base, length, width, and curb_weight, while height shows
↳ weak relationships.

# Group 2: Engine and performance characteristics
# The pair plot shows that price is strongly positively related to engine_size and horsepower
# Bore shows a moderate positive relationship with price, while stroke and compression_ratio exhibit weak
↳ or unclear relationships.
# Additionally, engine_size, horsepower, and bore are highly correlated with each other, suggesting
↳ potential multicollinearity among engine-related variables.

# Group 3: Fuel efficiency and risk-related variables

```

```

# The pair plot shows a strong negative relationship between price and fuel efficiency variables
↳ (city_mpg and highway_mpg).
# City_mpg and highway_mpg are highly correlated with each other, suggesting potential multicollinearity.
↳ In contrast, peak_rpm and normalized_losses show weak relationships with price.

# Categorical Variables
# The boxplots indicate that price varies across several categorical variables, with strong effects from
↳ make, body_style, drive_wheels, engine_location, and num_cylinders.
# Engine_type and fuel_system show moderate differences, while aspiration has a smaller effect, and
↳ symboling, num_doors, and fuel_type appear to have weaker relationships with price.

```

Correlation Analysis

```

# Compute correlation matrix
cor_matrix <- cor(num_vars)

# Correlation with response (price)
kable(round(cor_matrix[, "price"], 2))

```

	x
normalized_losses	0.10
wheel_base	0.58
length	0.69
width	0.75
height	0.13
curb_weight	0.83
engine_size	0.87
bore	0.54
stroke	0.08
compression_ratio	0.08
horsepower	0.81
peak_rpm	-0.10
city_mpg	-0.69
highway_mpg	-0.71
price	1.00

```

# Observations:
# The correlation analysis shows that price is strongly positively correlated with engine_size,
↳ horsepower, and curb_weight.

# Vehicle size variables such as width, length, and wheel_base also show moderate to strong positive
↳ correlations with price.

# In contrast, fuel efficiency variables (city_mpg and highway_mpg) are strongly negatively correlated
↳ with price.

# Other variables such as normalized_losses, height, stroke, compression_ratio, and peak_rpm show weak
↳ correlations with price.

```

Multicollinearity Check

```
# Variables suspected to be highly correlated
suspect_vars <- c("wheel_base", "length", "width", "curb_weight",
                 "engine_size", "horsepower", "bore",
                 "city_mpg", "highway_mpg")

options(width = 150)
kable(round(cor_matrix[suspect_vars, suspect_vars], 2))
```

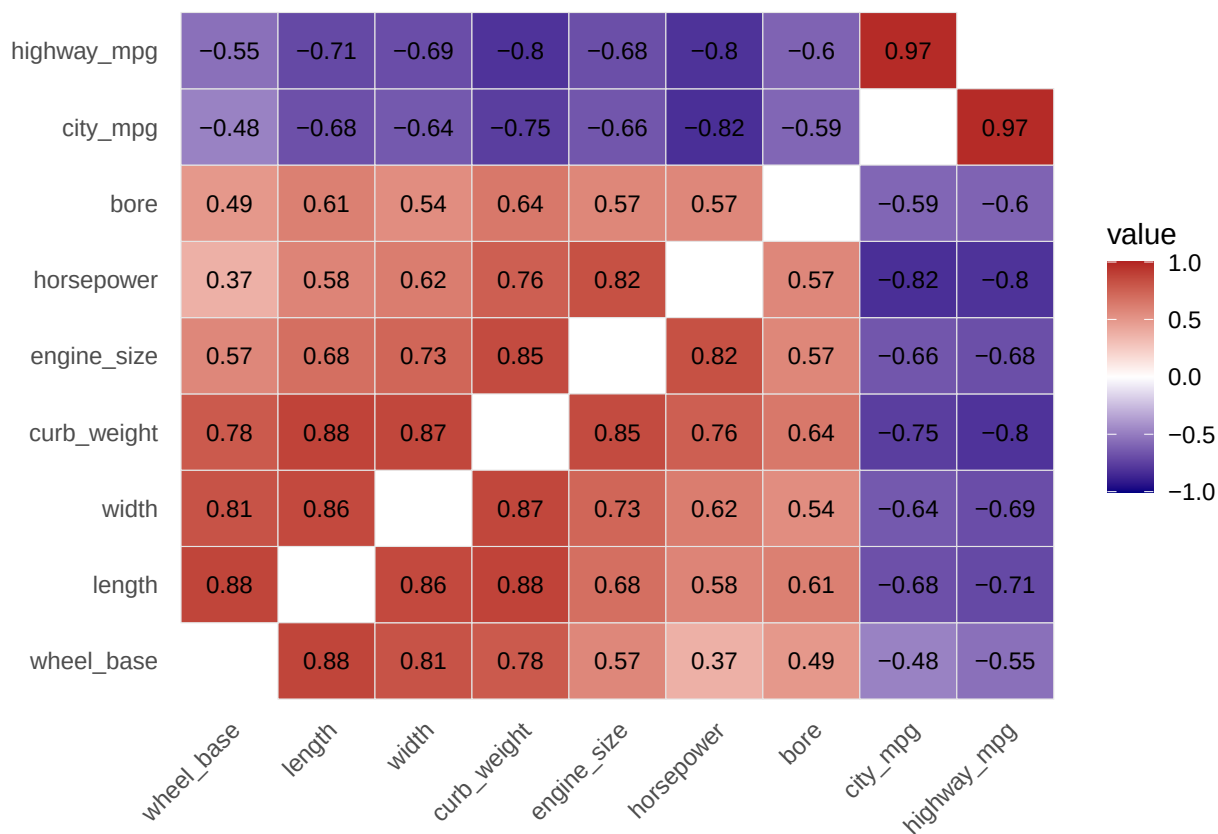
	wheel_base	length	width	curb_weight	engine_size	horsepower	bore	city_mpg	highway_mpg
wheel_base	1.00	0.88	0.81	0.78	0.57	0.37	0.49	-0.48	-0.55
length	0.88	1.00	0.86	0.88	0.68	0.58	0.61	-0.68	-0.71
width	0.81	0.86	1.00	0.87	0.73	0.62	0.54	-0.64	-0.69
curb_weight	0.78	0.88	0.87	1.00	0.85	0.76	0.64	-0.75	-0.80
engine_size	0.57	0.68	0.73	0.85	1.00	0.82	0.57	-0.66	-0.68
horsepower	0.37	0.58	0.62	0.76	0.82	1.00	0.57	-0.82	-0.80
bore	0.49	0.61	0.54	0.64	0.57	0.57	1.00	-0.59	-0.60
city_mpg	-0.48	-0.68	-0.64	-0.75	-0.66	-0.82	-0.59	1.00	0.97
highway_mpg	-0.55	-0.71	-0.69	-0.80	-0.68	-0.80	-0.60	0.97	1.00

```
# Correlation heatmap with enhanced color contrast to identify multicollinearity.
library(ggplot2)
library(reshape2)

corr_data <- cor_matrix[suspect_vars, suspect_vars]
diag(corr_data) <- NA

melted_corr <- melt(corr_data, na.rm = TRUE)

ggplot(melted_corr, aes(x = Var1, y = Var2, fill = value)) +
  geom_tile(color = "grey90") +
  geom_text(aes(label = round(value, 2)), size = 3) +
  scale_fill_gradient2(
    low = "navy",
    mid = "white",
    high = "firebrick",
    midpoint = 0,
    limits = c(-1, 1)
  ) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1),
    panel.grid = element_blank(),
    axis.title = element_blank()
  )
```



```
# Observations
# The correlation heatmap reveals several strong relationships among the predictors. Size-related
↪ variables, including wheel_base, length, width, and curb_weight, are highly positively correlated
↪ (often above 0.8), indicating that they capture similar information about vehicle size.

# Engine-related variables such as engine_size and horsepower also show strong positive correlations, and
↪ both are highly correlated with curb_weight, suggesting that larger and heavier vehicles tend to have
↪ more powerful engines.

# In addition, city_mpg and highway_mpg are almost perfectly correlated (cor 0.97), and both are
↪ strongly negatively correlated with engine_size, horsepower, and curb_weight, indicating that more
↪ powerful and heavier vehicles are less fuel-efficient.

# Overall, these patterns suggest substantial multicollinearity among predictors, and therefore, only a
↪ subset of these highly correlated variables should be included in the regression model to avoid
↪ redundancy and instability in coefficient estimates.
```

ANOVA for Categorical Variables

```
# Perform one-way ANOVA for each categorical variable to assess its relationship with the response
↪ variable
find_significant_cat <- function(data, response, alpha = 0.05) {

  # Identify categorical variables
  cat_vars <- names(data)[sapply(data, is.factor)]

  # Store results
  results <- data.frame(Variable = character(), P_value = numeric())

  # Loop through variables and extract p-values
```

```

for (var in cat_vars) {
  fit <- lm(as.formula(paste(response, "~", var)), data = data)
  p <- anova(fit)[1, "Pr(>F)"]

  results <- rbind(results, data.frame(Variable = var, P_value = p))
}

# Sort and split results
results <- results[order(results$P_value), ]

return(list(
  all_results = results,
  significant = subset(results, P_value < alpha),
  non_significant = subset(results, P_value >= alpha)
))
}

# Apply function
res <- find_significant_cat(auto_clean, "price")

# Print non-significant variables (p >= 0.05)
print(res$non_significant$Variable)

```

```
[1] "fuel_type" "num_doors"
```

```

# Observations:
# The ANOVA results indicate that most categorical variables have a significant relationship with price.
↪ However, fuel type and number of doors are not statistically significant (p > 0.05), suggesting that
↪ they do not have a meaningful effect on price. Therefore, these variables may not be important
↪ predictors in the model.

# Decision Point 1 (Lei, Catherine, Jessica, 2026-03-30):
# Based on the EDA, correlation analysis, and ANOVA results, we agreed to remove two categorical
↪ variables (fuel_type and num_doors) due to their lack of statistical significance (p-value > 0.05).
↪ Additionally, five continuous variables were excluded because they exhibited weak relationships with
↪ the response variable (price).

```

Model Fitting

Fit a multiple linear regression model and generate diagnostic plots to assess model assumptions.

```

# Fit a multiple linear regression model using all predictors
fit_model <- lm(price ~ . - fuel_type - num_doors - normalized_losses - height - stroke -
  ↪ compression_ratio - peak_rpm, data = auto_clean)

# Generate regression diagnostic plots:

# Residuals vs Fitted
# Compute studentized residuals and fitted values for outlier detection
stud_res <- rstudent(fit_model)
fitted_vals <- fitted(fit_model)

# Plot studentized residuals against fitted values to identify potential outliers
plot(fitted_vals, stud_res,
  xlab = "Fitted Values",
  ylab = "Studentized Residuals")

```

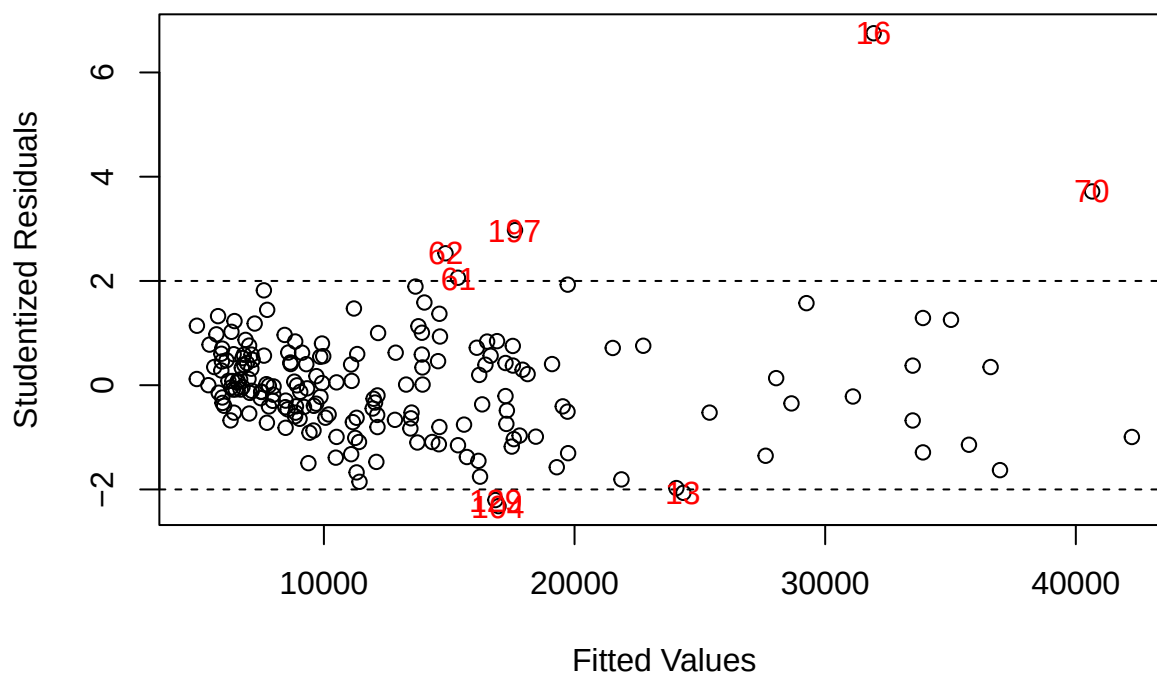
```

# Add reference lines at  $\pm 2$  to highlight potential outliers
abline(h = c(-2, 2), lty = 2)

# Identify observations with large studentized residuals ( $|res| > 2$ )
outliers <- which(abs(stud_res) > 2)

# Label the detected outliers on the plot (with jitter to reduce overlap)
text(jitter(fitted_vals[outliers]),
     jitter(stud_res[outliers]),
     labels = outliers,
     col = "red")

```



```

# Print indices of potential outliers
which(abs(stud_res) > 2)

```

```

14 17 66 67 75 109 135 203
13 16 61 62 70 104 129 197

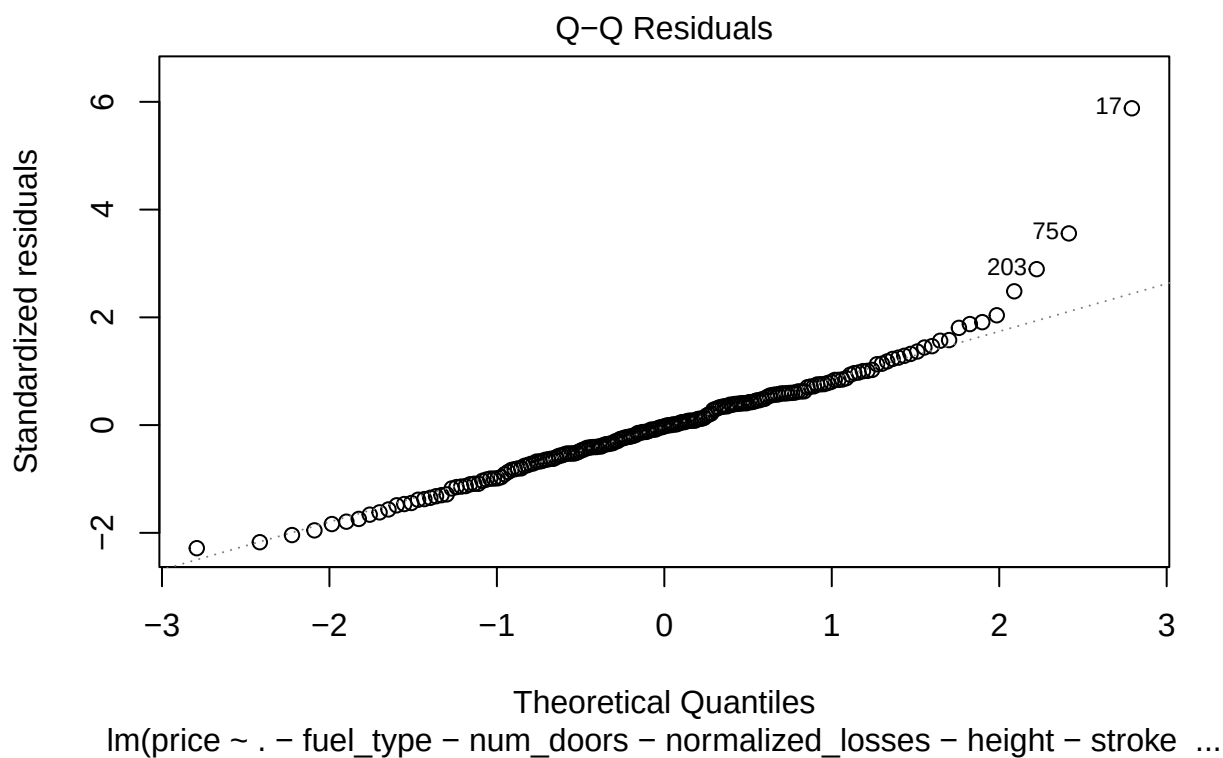
```

```

# Normal Q-Q Plot
plot(fit_model, which = 2)

```

Warning: not plotting observations with leverage one:
 18, 28, 42, 43, 46, 55, 71, 121

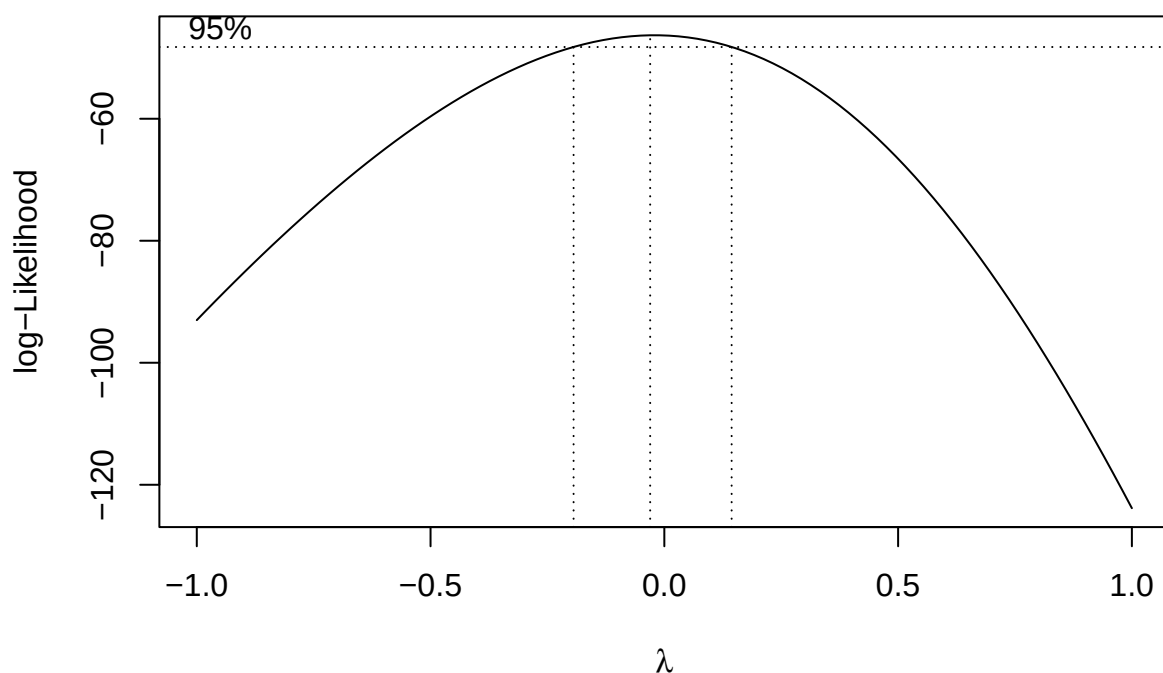


```
# Decision Point 2 (Lei, Catherine, Jessica, 2026-03-30):
# The Residuals vs Fitted plot shows increasing variability. The normal Q-Q plot shows the deviation from
↳ the reference line in the tails, indicating a skewed distribution of residuals. This suggests that a
↳ transformation of the response variable may be appropriate.
```

Transformation

Apply the Box-Cox procedure to identify an appropriate transformation for the response variable (price) by evaluating different values of λ .

```
# Box-Cox to find optimal transformation
library(MASS)
boxcox(fit_model, lambda=seq(-1, 1, 1/20))
```



```
# Refit the model using log(price) as the response variable
# -----
# Step 1: Baseline model
# -----
refit_model_1 <- lm(
  log(price) ~ .
  - fuel_type - num_doors - normalized_losses - height - stroke - compression_ratio - peak_rpm,
  data = auto_clean
)

# -----
# Step 2: Function to test quadratic term
# -----
test_quad <- function(var) {
  formula_quad <- as.formula(paste(". ~ . + I(", var, "^2)", sep = ""))

  model_quad <- update(refit_model_1, formula_quad)
  test <- anova(refit_model_1, model_quad)

  p_value <- test$`Pr(>F)`[2]

  return(data.frame(
    Variable = var,
    p_value = p_value
  ))
}

# -----
# Step 3: Variables to test
# -----
vars <- c("horsepower", "engine_size", "curb_weight",
          "length", "width", "bore",
          "city_mpg", "highway_mpg")

# Run tests
results <- do.call(rbind, lapply(vars, test_quad))
```

```

# Add significance column
results$Significant <- results$p_value < 0.05

# Show results
print(results)

```

	Variable	p_value	Significant
1	horsepower	0.3331121	FALSE
2	engine_size	0.1914933	FALSE
3	curb_weight	0.4726680	FALSE
4	length	0.6681523	FALSE
5	width	0.9054023	FALSE
6	bore	0.3323182	FALSE
7	city_mpg	0.1179347	FALSE
8	highway_mpg	0.1069329	FALSE

```

# -----
# Step 4: Keep significant quadratic terms
# -----
sig_vars <- results$Variable[results$Significant]

quad_terms <- paste0("I(", sig_vars, "^2)", collapse = " + ")

# -----
# Step 5: Final model
# -----
if (length(sig_vars) > 0) {

  quad_terms <- paste0("I(", sig_vars, "^2)", collapse = " + ")

  final_formula <- as.formula(
    paste("log(price) ~ . - fuel_type - num_doors - normalized_losses - height - stroke -
      ↪ compression_ratio - peak_rpm +", quad_terms)
  )

} else {

  final_formula <- as.formula(
    "log(price) ~ . - fuel_type - num_doors - normalized_losses - height - stroke - compression_ratio -
      ↪ peak_rpm"
  )

}

final_model <- lm(final_formula, data = auto_clean)
print(results)

```

	Variable	p_value	Significant
1	horsepower	0.3331121	FALSE
2	engine_size	0.1914933	FALSE
3	curb_weight	0.4726680	FALSE
4	length	0.6681523	FALSE

```

5      width 0.9054023      FALSE
6      bore  0.3323182      FALSE
7      city_mpg 0.1179347    FALSE
8      highway_mpg 0.1069329  FALSE

```

Decision Point 3 (Lei, Catherine, Jessica, 2026-03-30):

Based on the plots in the EDA part, we observed evidence of non-constant variance in the residuals, and
↪ the response variable (price) was right-skewed. Therefore, a Box-Cox analysis was conducted to assess
↪ an appropriate transformation. The estimated lambda was close to 0, suggesting that a log
↪ transformation is suitable. As a result, we agreed to refit the model using log(price) to stabilize
↪ variance and improve the normality of residuals.

After applying the transformation, potential nonlinear relationships were further assessed by testing
↪ quadratic terms for selected predictors. ANOVA tests show that none of the quadratic terms are
↪ statistically significant ($p > 0.05$), suggesting that a linear specification is sufficient. Including
↪ higher-order terms does not improve model fit and may introduce unnecessary complexity.

Model Fitting (Con't)

Fit a multiple linear regression model and generate diagnostic plots to assess model assumptions.

Generate regression diagnostic plots:

Residuals vs Fitted

Compute studentized residuals and fitted values for outlier detection

```
stud_res <- rstudent(refit_model_1)
```

```
fitted_vals <- fitted(refit_model_1)
```

Plot studentized residuals against fitted values to identify potential outliers

```
plot(fitted_vals, stud_res,
     xlab = "Fitted Values",
     ylab = "Studentized Residuals")
```

Add reference lines at ± 2 to highlight potential outliers

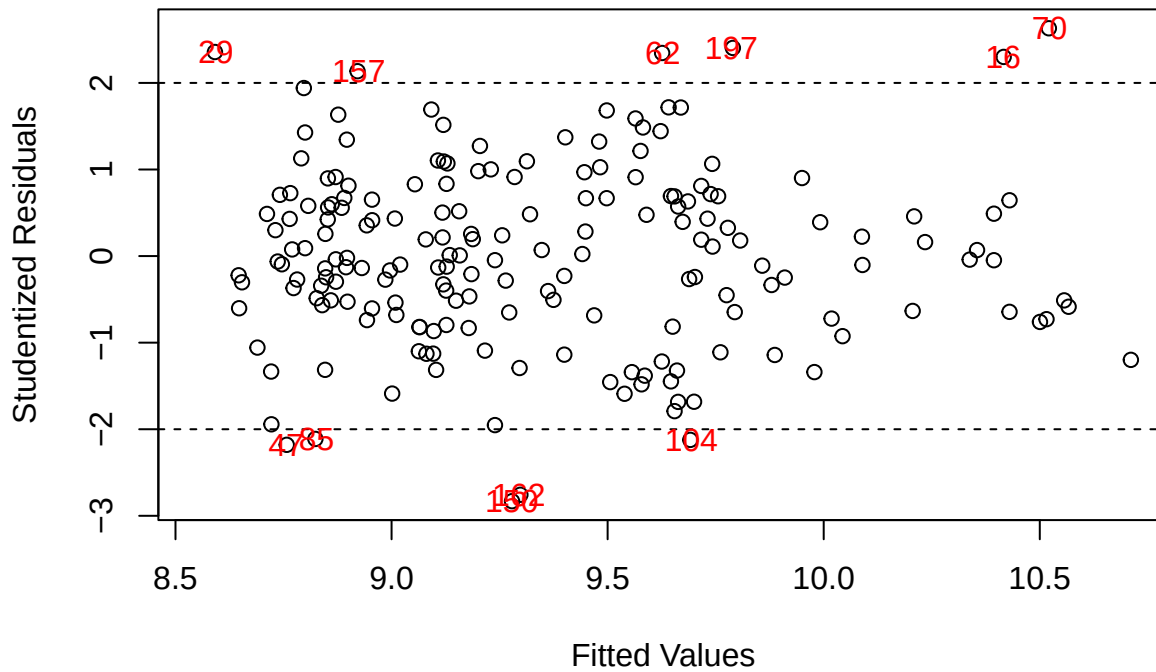
```
abline(h = c(-2, 2), lty = 2)
```

Identify observations with large studentized residuals ($|res| > 2$)

```
outliers <- which(abs(stud_res) > 2)
```

Label the detected outliers on the plot (with jitter to reduce overlap)

```
text(jitter(fitted_vals[outliers]),
     jitter(stud_res[outliers]),
     labels = outliers,
     col = "red")
```



```
# Print indices of potential outliers
which(abs(stud_res) > 2)
```

```
17 31 51 67 75 90 109 156 163 168 203
16 29 47 62 70 85 104 150 157 162 197
```

```
# Split variables into three groups for clearer table presentation
n <- ncol(auto_clean)
cols1 <- 1:floor(n/3)
cols2 <- (floor(n/3)+1):floor(2*n/3)
cols3 <- (floor(2*n/3)+1):n

# Display outlier observations across three tables to improve readability
kable(auto_clean[outliers, cols1], caption = "Outliers (Part 1)")
```

Table 5: Outliers (Part 1)

	symboling	normalized_losses	make	fuel_type	aspiration	num_doors	body_style	drive_wheels
17	0	115	bmw	gas	std	two	sedan	rwd
31	2	137	honda	gas	std	two	hatchback	fwd
51	1	104	mazda	gas	std	two	hatchback	fwd
67	0	115	mazda	diesel	std	four	sedan	rwd
75	1	115	mercedes-benz	gas	std	two	hardtop	rwd
90	1	128	nissan	gas	std	two	sedan	fwd
109	0	161	peugot	diesel	turbo	four	sedan	rwd
156	0	91	toyota	gas	std	four	wagon	4wd
163	0	91	toyota	gas	std	four	sedan	fwd
168	2	134	toyota	gas	std	two	hardtop	rwd
203	-1	95	volvo	gas	std	four	sedan	rwd

```
kable(auto_clean[outliers, cols2], caption = "Outliers (Part 2)")
```

Table 6: Outliers (Part 2)

	engine_location	wheel_base	length	width	height	curb_weight	engine_type	num_cylinders	engine_size
17	front	103.5	193.8	67.9	53.7	3380	ohc	six	209
31	front	86.6	144.6	63.9	50.8	1713	ohc	four	92
51	front	93.1	159.1	64.2	54.1	1890	ohc	four	91
67	front	104.9	175.0	66.1	54.4	2700	ohc	four	134
75	front	112.0	199.2	72.0	55.4	3715	ohcv	eight	304
90	front	94.5	165.3	63.8	54.5	1889	ohc	four	97
109	front	107.9	186.7	68.4	56.7	3197	l	four	152
156	front	95.7	169.7	63.6	59.1	3110	ohc	four	92
163	front	95.7	166.3	64.4	52.8	2140	ohc	four	98
168	front	98.4	176.2	65.6	52.0	2540	ohc	four	146
203	front	109.1	188.8	68.9	55.5	3012	ohcv	six	173

```
kable(auto_clean[outliers, cols3], caption = "Outliers (Part 3)")
```

Table 7: Outliers (Part 3)

	fuel_system	bore	stroke	compression_ratio	horsepower	peak_rpm	city_mpg	highway_mpg	price
17	mpfi	3.62	3.39	8.0	182	5400	16	22	41315
31	1bbl	2.91	3.41	9.6	58	4800	49	54	6479
51	2bbl	3.03	3.15	9.0	68	5000	30	31	5195
67	idi	3.43	3.64	22.0	72	4200	31	39	18344
75	mpfi	3.80	3.35	8.0	184	4500	14	16	45400
90	2bbl	3.15	3.29	9.4	69	5200	31	37	5499
109	idi	3.70	3.52	21.0	95	4150	28	33	13200
156	2bbl	3.05	3.03	9.0	62	4800	27	32	8778
163	2bbl	3.19	3.03	9.0	70	4800	28	34	9258
168	mpfi	3.62	3.50	9.3	116	4800	24	30	8449
203	mpfi	3.58	2.87	8.8	134	5500	18	23	21485

```
# Check correlation across time
library(lmtest)
```

Loading required package: zoo

Attaching package: 'zoo'

The following objects are masked from 'package:base':

```
as.Date, as.Date.numeric
```

```
dwtest(refit_model_1)
```

Durbin-Watson test

```
data: refit_model_1
```

```
DW = 1.6592, p-value = 0.7877
```

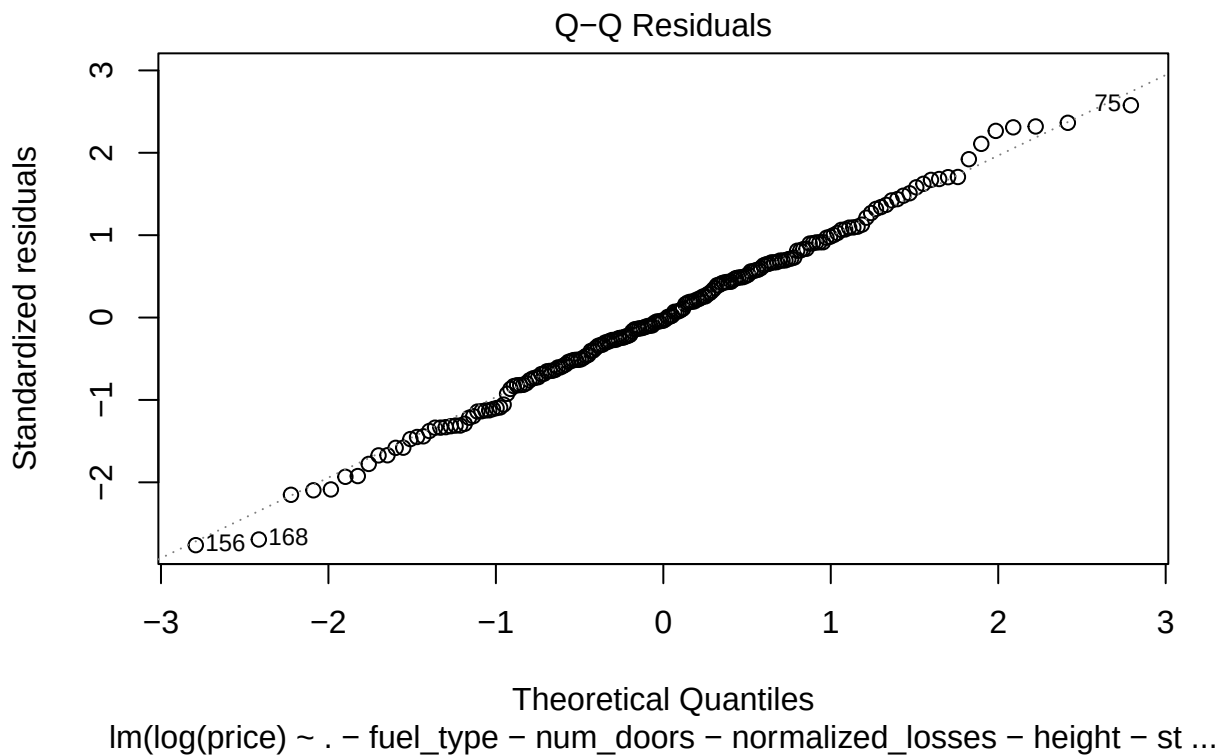
```
alternative hypothesis: true autocorrelation is greater than 0
```

```
# Normal Q-Q Plot
```

```
plot(refit_model_1, which = 2)
```

Warning: not plotting observations with leverage one:

18, 28, 42, 43, 46, 55, 71, 121



```
# Cook's Distance Plot
```

```
cooks <- cooks.distance(refit_model_1)
```

```
cooks_indexed <- data.frame(
```

```
  Obs = 1:length(cooks),
```

```
  Cook = cooks
```

```
)
```

```
vals <- paste(cooks_indexed$Obs, round(cooks_indexed$Cook, 4), sep = ": ")
```

```
vals <- c(vals, "")
```

```
cooks_mat <- matrix(vals, ncol = 5, byrow = TRUE)
```

```
kable(cooks_mat, caption = "Cook's Distance (Obs=Value)")
```

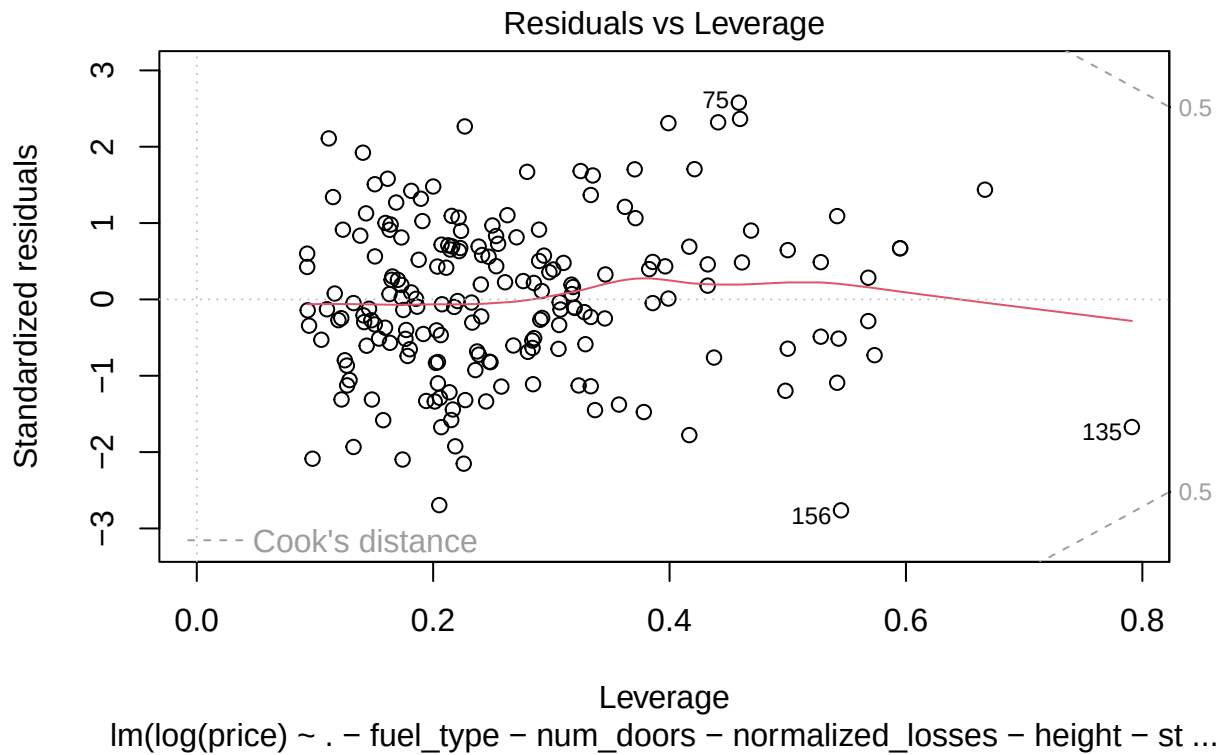
Table 8: Cook's Distance (Obs:Value)

1: 0.0382	2: 0.0058	3: 0.0701	4: 0.0112	5: 0.0021
6: 0.0017	7: 0.0077	8: 1e-04	9: 1e-04	10: 0.0017
11: 0.0031	12: 0.0028	13: 0.0045	14: 3e-04	15: 0
16: 0.0255	17: 0.0029	18: NaN	19: 0.0045	20: 0.0045
21: 5e-04	22: 3e-04	23: 0.0103	24: 4e-04	25: 0
26: 0.0076	27: 1e-04	28: NaN	29: 0.072	30: 0.0031
31: 0.0072	32: 0.0011	33: 2e-04	34: 0.001	35: 0
36: 0.0051	37: 0	38: 6e-04	39: 0.0231	40: 0.0018
41: 0.0018	42: NaN	43: NaN	44: 0.0071	45: 0.0071
46: NaN	47: 0.0229	48: 0.0041	49: 1e-04	50: 8e-04
51: 0.001	52: 0.011	53: 4e-04	54: 0.0158	55: NaN
56: 3e-04	57: 0.0097	58: 0.0068	59: 0	60: 0.0032
61: 0.029	62: 0.06	63: 0.0027	64: 2e-04	65: 0.0027
66: 0	67: 0.0077	68: 0.0122	69: 0.0241	70: 0.0954
71: NaN	72: 0.0023	73: 0	74: 8e-04	75: 0.0024
76: 2e-04	77: 0.0011	78: 0	79: 0.0069	80: 0.0042
81: 0.0116	82: 0	83: 0.001	84: 0	85: 0.008
86: 0	87: 2e-04	88: 0	89: 0.0036	90: 3e-04
91: 0.002	92: 6e-04	93: 0.0102	94: 0.0225	95: 0.003
96: 1e-04	97: 0.0097	98: 0.0037	99: 0.0068	100: 1e-04
101: 6e-04	102: 0.0025	103: 0.0079	104: 0.0157	105: 0.0076
106: 0.0087	107: 0.0027	108: 1e-04	109: 0.0093	110: 0.0019
111: 0.0082	112: 0.0023	113: 0.0023	114: 3e-04	115: 2e-04
116: 0	117: 0.0023	118: 0.0039	119: 3e-04	120: 0.0224
121: NaN	122: 0	123: 0.0026	124: 0.0053	125: 0.0239
126: 0.0239	127: 0.0181	128: 0.0031	129: 0.1794	130: 0.0184
131: 0.001	132: 0.0113	133: 0.0176	134: 0.0018	135: 0.0017
136: 0.002	137: 2e-04	138: 0.0011	139: 0.004	140: 0.0017
141: 1e-04	142: 0.0055	143: 0.0038	144: 0.0058	145: 0.0028
146: 2e-04	147: 0	148: 2e-04	149: 0.0042	150: 0.155
151: 6e-04	152: 0	153: 9e-04	154: 0.0019	155: 0.0022
156: 0.004	157: 0.0095	158: 8e-04	159: 0.002	160: 3e-04
161: 0.0018	162: 0.0318	163: 0.0073	164: 0.0016	165: 7e-04
166: 0	167: 0.0359	168: 0.001	169: 4e-04	170: 2e-04
171: 0.0032	172: 0.0055	173: 0.0023	174: 4e-04	175: 0.0083
176: 5e-04	177: 0.0052	178: 0.0031	179: 0.0029	180: 0.0019
181: 0.0015	182: 0	183: 0.0019	184: 0.0034	185: 0.0074
186: 0.0112	187: 0.0053	188: 0.0056	189: 0.0179	190: 0.0124
191: 0.0141	192: 0.0023	193: 4e-04	194: 8e-04	195: 8e-04
196: 0.0098	197: 0.0806	198: 0.0122	199: 0.0011	

```
plot(refit_model_1, which = 5)
```

Warning: not plotting observations with leverage one:

18, 28, 42, 43, 46, 55, 71, 121



```
# Observations with Cook's D > 0.5 or D > 1
which(cooks > 0.5)
```

```
named integer(0)
```

```
which(cooks > 1)
```

```
named integer(0)
```

```
# Observations:
# After applying the log transformation to the response variable, the model diagnostics show substantial
  ↳ improvement.

# The residuals vs. fitted plot shows improved spread of residuals, with less evidence of increasing
  ↳ variability. This suggests that the issue of non-constant variance has been mitigated.

# To assess the independence of residuals, the Durbin-Watson test was conducted. The result indicates no
  ↳ evidence of autocorrelation in the residuals, suggesting that the independence assumption is
  ↳ satisfied.

# The normal Q-Q plot shows that residuals are now more closely aligned with the reference line, although
  ↳ slight deviations remain in the tails, suggesting minor departures from normality.

# In addition, Cook's distance indicates that there are no highly influential observations.

# Overall, the transformation has effectively improved model assumptions, and the remaining deviations
  ↳ are not severe enough to invalidate the model.

# Decision Point 4 (Lei, Catherine, Jessica, 2026-03-30):
```

Regarding potential outliers, the dataset documentation does not provide specific explanations for their presence. Therefore, we decided to retain these observations and avoid making unsupported assumptions about their validity.

Model Selection

Variables were removed based on alias results, high cardinality, and high VIF to ensure a more stable and reliable model.

```
# Check for perfect multicollinearity using alias()
# The output indicates that some categorical variables (e.g., fuel_system and engine_type) are perfectly
# collinear with other predictors such as num_cylinders.
alias_indexed <- as.data.frame(alias(refit_model_1)$Complete)
split_cols <- split(seq_len(ncol(alias_indexed)),
                    ceiling(seq_len(ncol(alias_indexed)) / 4))
for (cols in split_cols) {
  print(knitr::kable(alias_indexed[, cols, drop = FALSE]))
}
```

	(Intercept)	symboling-2	symboling0	symboling1
engine_typeohcf	0	0	0	0
num_cylindersthree	0	0	0	0
num_cylinderstwelve	1	0	0	0

	symboling2	symboling3	makeaudi	makebmw
engine_typeohcf	0	0	0	0
num_cylindersthree	0	0	0	0
num_cylinderstwelve	0	0	0	0

	makechevrolet	makedodge	makehonda	makeisuzu
engine_typeohcf	0	0	0	0
num_cylindersthree	0	0	0	0
num_cylinderstwelve	0	0	0	0

	makejaguar	makemazda	makemercedes-benz	makemercury
engine_typeohcf	0	0	0	0
num_cylindersthree	0	0	0	0
num_cylinderstwelve	0	0	0	0

	makemitsubishi	makenissan	makepeugot	makeplymouth
:	:	:	:	:
engine_typeohcf	0	0	0	0
num_cylinderehree	0	0	-1	0
num_cylinderehtwelve	0	0	1	0

	makeporsche	makerenault	makesaab	makesubaru
:	:	:	:	:
engine_typeohcf	0	0	0	1
num_cylinderehree	0	0	0	0
num_cylinderehtwelve	0	0	0	0

	maketoyota	makevolkswagen	makevolvo	aspirationturbo
:	:	:	:	:
engine_typeohcf	0	0	0	0
num_cylinderehree	0	0	0	0
num_cylinderehtwelve	0	0	0	0

	body_stylehardtop	body_stylehatchback	body_stylesedan	body_stylewagon
:	:	:	:	:
engine_typeohcf	0	0	0	0
num_cylinderehree	0	0	0	0
num_cylinderehtwelve	0	0	0	0

	drive_wheelsfwd	drive_wheelsrwd	engine_locationrear	wheel_base
:	:	:	:	:
engine_typeohcf	0	0	1	0
num_cylinderehree	0	0	0	0
num_cylinderehtwelve	0	0	0	0

	length	width	curb_weight	engine_typeel
:	:	:	:	:
engine_typeohcf	0	0	0	0
num_cylinderehree	0	0	0	1
num_cylinderehtwelve	0	0	0	-1

	engine_typeohc	engine_typeohcv	engine_typerotor	num_cylindersfour
:	:	:	:	:
engine_typeohcf	0	0	0	0
num_cylinderehree	0	0	0	0
num_cylinderehtwelve	0	0	-1	-1

	num_cylindersfive	num_cylinderssix	num_cylinderseight	engine_size
engine_typeohcf	0	0	0	0
num_cylindersthree	0	0	0	0
num_cylinderstwelve	-1	-1	-1	0

	fuel_system2bbl	fuel_system4bbl	fuel_systemmidi	fuel_systemmfi
engine_typeohcf	0	0	0	0
num_cylindersthree	0	0	0	0
num_cylinderstwelve	0	0	0	0

	fuel_systemmpfi	fuel_systemspdi	fuel_systemspfi	bore
engine_typeohcf	0	0	0	0
num_cylindersthree	0	0	0	0
num_cylinderstwelve	0	0	0	0

	horsepower	city_mpg	highway_mpg
engine_typeohcf	0	0	0
num_cylindersthree	0	0	0
num_cylinderstwelve	0	0	0

```
# Refit the model after removing variables that exhibit perfect multicollinearity and variables with many
↳ levels (e.g., make) that may introduce instability.
```

```
refit_model_2 <- lm(log(price) ~ . - fuel_type - num_doors - normalized_losses - height - stroke -
↳ compression_ratio - peak_rpm - make - engine_type, data = auto_clean)
```

```
# Refit the model after addressing multicollinearity issues identified from both alias and VIF
↳ diagnostics.
```

```
library(car)
```

Loading required package: carData

```
vif(refit_model_2)
```

	GVIF	Df	GVIF ^{1/(2*Df)}
symboling	13.079585	5	1.293181
aspiration	4.230152	1	2.056733
body_style	8.686513	4	1.310255
drive_wheels	5.553197	2	1.535097
engine_location	2.181012	1	1.476825
wheel_base	11.681519	1	3.417824

length	14.387351	1	3.793066
width	9.608956	1	3.099832
curb_weight	29.359290	1	5.418421
num_cylinders	241.882006	6	1.579915
engine_size	28.762196	1	5.363040
fuel_system	174.972339	7	1.446146
bore	4.943383	1	2.223372
horsepower	22.189280	1	4.710550
city_mpg	33.952622	1	5.826888
highway_mpg	32.760941	1	5.723717

```
# Decision Point 5 (Lei, Catherine, Jessica, 2026-03-30):
# engine_type is removed due to perfect multicollinearity identified by alias.
# make is removed due to high cardinality and potential instability. (The variable make has many
  ↳ categories, creating many dummy variables with few observations, which may lead to unstable
  ↳ estimates.)
# curb_weight, engine_size, city_mpg and highway_mpg's VIFs are larger than 5 but lower than 10, so we
  ↳ agreed to keep them for now.
```

Stepwise regression, forward selection and backward elimination methods are used to select the optimal model.

```
# Stepwise Regression Method
null_model <- lm(log(price) ~ 1, data = auto_clean)
full_model <- refit_model_2
step_model <- step(null_model,
  scope = list(lower = null_model, upper = full_model),
  direction = "both")
```

Start: AIC=-271.45

```
log(price) ~ 1
```

	Df	Sum of Sq	RSS	AIC
+ curb_weight	1	40.003	10.356	-584.18
+ horsepower	1	35.149	15.210	-507.70
+ engine_size	1	34.661	15.698	-501.42
+ width	1	32.085	18.274	-471.18
+ highway_mpg	1	31.279	19.081	-462.58
+ city_mpg	1	30.697	19.663	-456.60
+ length	1	30.404	19.955	-453.66
+ num_cylinders	6	26.744	23.615	-410.15
+ fuel_system	7	26.357	24.002	-404.92
+ drive_wheels	2	23.760	26.600	-394.47
+ wheel_base	1	20.303	30.056	-372.16
+ bore	1	18.432	31.928	-360.14
+ symboling	5	11.773	38.587	-314.44
+ body_style	4	7.509	42.850	-295.58
+ engine_location	1	3.660	46.699	-284.46
+ aspiration	1	3.580	46.779	-284.12
<none>			50.359	-271.45

Step: AIC=-584.18

log(price) ~ curb_weight

	Df	Sum of Sq	RSS	AIC
+ horsepower	1	2.983	7.373	-649.80
+ engine_location	1	2.541	7.816	-638.19
+ body_style	4	2.118	8.239	-621.71
+ num_cylinders	6	1.957	8.400	-613.85
+ city_mpg	1	1.365	8.991	-610.32
+ fuel_system	7	1.849	8.508	-609.32
+ engine_size	1	0.965	9.391	-601.65
+ highway_mpg	1	0.788	9.569	-597.93
+ symboling	5	1.103	9.253	-596.60
+ drive_wheels	2	0.780	9.576	-595.77
+ wheel_base	1	0.492	9.865	-591.86
+ width	1	0.136	10.221	-584.81
<none>			10.356	-584.18
+ bore	1	0.084	10.272	-583.81
+ aspiration	1	0.053	10.303	-583.20
+ length	1	0.018	10.339	-582.52
- curb_weight	1	40.003	50.359	-271.45

Step: AIC=-649.8

log(price) ~ curb_weight + horsepower

	Df	Sum of Sq	RSS	AIC
+ body_style	4	1.1844	6.1886	-676.65
+ engine_location	1	0.7894	6.5836	-670.34
+ fuel_system	7	1.1201	6.2529	-668.59
+ num_cylinders	6	1.0331	6.3400	-667.84
+ width	1	0.3400	7.0330	-657.20
+ drive_wheels	2	0.3423	7.0308	-655.26
+ length	1	0.1319	7.2411	-651.39
+ wheel_base	1	0.0794	7.2937	-649.95
<none>			7.3730	-649.80
+ aspiration	1	0.0538	7.3193	-649.26
+ city_mpg	1	0.0425	7.3305	-648.95
+ engine_size	1	0.0085	7.3645	-648.03
+ highway_mpg	1	0.0006	7.3724	-647.82
+ bore	1	0.0001	7.3729	-647.80
+ symboling	5	0.1089	7.2641	-642.76
- horsepower	1	2.9835	10.3565	-584.18
- curb_weight	1	7.8371	15.2102	-507.70

Step: AIC=-676.65

log(price) ~ curb_weight + horsepower + body_style

	Df	Sum of Sq	RSS	AIC
+ num_cylinders	6	0.9343	5.2543	-697.22
+ engine_location	1	0.5264	5.6622	-692.34
+ fuel_system	7	0.7357	5.4530	-687.83
+ width	1	0.3650	5.8236	-686.75
+ wheel_base	1	0.1803	6.0083	-680.53
+ length	1	0.1410	6.0476	-679.24
+ drive_wheels	2	0.1833	6.0053	-678.63
+ city_mpg	1	0.0713	6.1173	-676.95
<none>			6.1886	-676.65
+ engine_size	1	0.0308	6.1578	-675.64
+ highway_mpg	1	0.0144	6.1742	-675.11
+ aspiration	1	0.0104	6.1782	-674.98
+ bore	1	0.0073	6.1813	-674.88
+ symboling	5	0.1857	6.0029	-672.71
- body_style	4	1.1844	7.3730	-649.80
- horsepower	1	2.0501	8.2387	-621.71
- curb_weight	1	6.2334	12.4220	-539.99

Step: AIC=-697.22

log(price) ~ curb_weight + horsepower + body_style + num_cylinders

	Df	Sum of Sq	RSS	AIC
+ engine_location	1	0.4492	4.8051	-713.00
+ fuel_system	7	0.4958	4.7585	-702.94
+ width	1	0.1839	5.0705	-702.31
+ wheel_base	1	0.1601	5.0942	-701.38
+ drive_wheels	2	0.2042	5.0501	-701.11
+ length	1	0.1476	5.1067	-700.89
<none>			5.2543	-697.22
+ highway_mpg	1	0.0248	5.2295	-696.16
+ bore	1	0.0216	5.2327	-696.04
+ aspiration	1	0.0022	5.2521	-695.30
+ city_mpg	1	0.0009	5.2534	-695.25
+ engine_size	1	0.0003	5.2540	-695.23
+ symboling	5	0.0956	5.1588	-690.87
- num_cylinders	6	0.9343	6.1886	-676.65
- body_style	4	1.0857	6.3400	-667.84
- horsepower	1	1.5552	6.8096	-647.62
- curb_weight	1	4.2028	9.4571	-582.26

Step: AIC=-713

log(price) ~ curb_weight + horsepower + body_style + num_cylinders +
engine_location

	Df	Sum of Sq	RSS	AIC
+ width	1	0.2151	4.5900	-720.11

+ wheel_base	1	0.1979	4.6072	-719.37
+ length	1	0.1903	4.6148	-719.04
+ fuel_system	7	0.4457	4.3593	-718.38
+ drive_wheels	2	0.1936	4.6114	-717.19
<none>			4.8051	-713.00
+ city_mpg	1	0.0168	4.7882	-711.70
+ bore	1	0.0026	4.8025	-711.11
+ aspiration	1	0.0020	4.8031	-711.09
+ engine_size	1	0.0017	4.8033	-711.07
+ highway_mpg	1	0.0010	4.8041	-711.04
+ symboling	5	0.0455	4.7596	-704.90
- engine_location	1	0.4492	5.2543	-697.22
- num_cylinders	6	0.8572	5.6622	-692.34
- body_style	4	0.8718	5.6769	-687.82
- horsepower	1	0.8864	5.6915	-681.31
- curb_weight	1	4.6481	9.4531	-580.34

Step: AIC=-720.11

log(price) ~ curb_weight + horsepower + body_style + num_cylinders +
engine_location + width

	Df	Sum of Sq	RSS	AIC
+ drive_wheels	2	0.22247	4.3676	-726.00
+ fuel_system	7	0.34347	4.2466	-721.59
+ wheel_base	1	0.07505	4.5150	-721.40
+ length	1	0.07313	4.5169	-721.31
<none>			4.5900	-720.11
+ city_mpg	1	0.02555	4.5645	-719.23
+ engine_size	1	0.00149	4.5885	-718.18
+ aspiration	1	0.00105	4.5890	-718.16
+ bore	1	0.00026	4.5898	-718.13
+ highway_mpg	1	0.00007	4.5899	-718.12
- width	1	0.21505	4.8051	-713.00
+ symboling	5	0.04874	4.5413	-712.24
- num_cylinders	6	0.60544	5.1955	-707.46
- engine_location	1	0.48043	5.0705	-702.31
- body_style	4	0.87275	5.4628	-693.47
- horsepower	1	0.84600	5.4360	-688.45
- curb_weight	1	1.42988	6.0199	-668.15

Step: AIC=-726

log(price) ~ curb_weight + horsepower + body_style + num_cylinders +
engine_location + width + drive_wheels

	Df	Sum of Sq	RSS	AIC
+ length	1	0.08412	4.2834	-727.87
<none>			4.3676	-726.00

+ city_mpg	1	0.03462	4.3329	-725.59
+ wheel_base	1	0.03184	4.3357	-725.46
+ bore	1	0.00656	4.3610	-724.30
+ fuel_system	7	0.26117	4.1064	-724.27
+ aspiration	1	0.00041	4.3671	-724.02
+ engine_size	1	0.00005	4.3675	-724.00
+ highway_mpg	1	0.00001	4.3675	-724.00
+ symboling	5	0.12240	4.2452	-721.66
- drive_wheels	2	0.22247	4.5900	-720.11
- width	1	0.24389	4.6114	-717.19
- num_cylinders	6	0.53017	4.8977	-715.20
- engine_location	1	0.46518	4.8327	-707.86
- body_style	4	0.75627	5.1238	-702.22
- curb_weight	1	0.68980	5.0574	-698.82
- horsepower	1	0.86765	5.2352	-691.94

Step: AIC=-727.87

```
log(price) ~ curb_weight + horsepower + body_style + num_cylinders +
  engine_location + width + drive_wheels + length
```

	Df	Sum of Sq	RSS	AIC
<none>			4.2834	-727.87
+ fuel_system	7	0.26898	4.0145	-726.78
+ aspiration	1	0.01268	4.2708	-726.46
+ bore	1	0.01157	4.2719	-726.41
+ city_mpg	1	0.01036	4.2731	-726.35
+ highway_mpg	1	0.00528	4.2782	-726.12
+ wheel_base	1	0.00516	4.2783	-726.11
- length	1	0.08412	4.3676	-726.00
+ engine_size	1	0.00009	4.2833	-725.88
- width	1	0.12704	4.4105	-724.05
+ symboling	5	0.12634	4.1571	-723.83
- drive_wheels	2	0.23345	4.5169	-721.31
- curb_weight	1	0.33248	4.6159	-715.00
- num_cylinders	6	0.57661	4.8600	-714.74
- engine_location	1	0.48622	4.7697	-708.47
- body_style	4	0.69760	4.9810	-705.85
- horsepower	1	0.84135	5.1248	-694.18

Forward Selection Method

```
null_model <- lm(log(price) ~ 1, data = auto_clean)
full_model <- refit_model_2
forward_model <- step(null_model,
  scope = list(lower = null_model, upper = full_model),
  direction = "forward")
```

Start: AIC=-271.45

```
log(price) ~ 1
```

	Df	Sum of Sq	RSS	AIC
+ curb_weight	1	40.003	10.356	-584.18
+ horsepower	1	35.149	15.210	-507.70
+ engine_size	1	34.661	15.698	-501.42
+ width	1	32.085	18.274	-471.18
+ highway_mpg	1	31.279	19.081	-462.58
+ city_mpg	1	30.697	19.663	-456.60
+ length	1	30.404	19.955	-453.66
+ num_cylinders	6	26.744	23.615	-410.15
+ fuel_system	7	26.357	24.002	-404.92
+ drive_wheels	2	23.760	26.600	-394.47
+ wheel_base	1	20.303	30.056	-372.16
+ bore	1	18.432	31.928	-360.14
+ symboling	5	11.773	38.587	-314.44
+ body_style	4	7.509	42.850	-295.58
+ engine_location	1	3.660	46.699	-284.46
+ aspiration	1	3.580	46.779	-284.12
<none>			50.359	-271.45

Step: AIC=-584.18

log(price) ~ curb_weight

	Df	Sum of Sq	RSS	AIC
+ horsepower	1	2.98346	7.3730	-649.80
+ engine_location	1	2.54053	7.8160	-638.19
+ body_style	4	2.11777	8.2387	-621.71
+ num_cylinders	6	1.95671	8.3998	-613.85
+ city_mpg	1	1.36535	8.9911	-610.32
+ fuel_system	7	1.84895	8.5075	-609.32
+ engine_size	1	0.96506	9.3914	-601.65
+ highway_mpg	1	0.78783	9.5687	-597.93
+ symboling	5	1.10342	9.2531	-596.60
+ drive_wheels	2	0.78047	9.5760	-595.77
+ wheel_base	1	0.49157	9.8649	-591.86
+ width	1	0.13569	10.2208	-584.81
<none>			10.3565	-584.18
+ bore	1	0.08414	10.2724	-583.81
+ aspiration	1	0.05304	10.3035	-583.20
+ length	1	0.01755	10.3390	-582.52

Step: AIC=-649.8

log(price) ~ curb_weight + horsepower

	Df	Sum of Sq	RSS	AIC
+ body_style	4	1.18442	6.1886	-676.65
+ engine_location	1	0.78941	6.5836	-670.34

+ fuel_system	7	1.12010	6.2529	-668.59
+ num_cylinders	6	1.03307	6.3400	-667.84
+ width	1	0.34003	7.0330	-657.20
+ drive_wheels	2	0.34226	7.0308	-655.26
+ length	1	0.13189	7.2411	-651.39
+ wheel_base	1	0.07936	7.2937	-649.95
<none>			7.3730	-649.80
+ aspiration	1	0.05377	7.3193	-649.26
+ city_mpg	1	0.04252	7.3305	-648.95
+ engine_size	1	0.00855	7.3645	-648.03
+ highway_mpg	1	0.00060	7.3724	-647.82
+ bore	1	0.00011	7.3729	-647.80
+ symboling	5	0.10895	7.2641	-642.76

Step: AIC=-676.65

log(price) ~ curb_weight + horsepower + body_style

	Df	Sum of Sq	RSS	AIC
+ num_cylinders	6	0.93431	5.2543	-697.22
+ engine_location	1	0.52638	5.6622	-692.34
+ fuel_system	7	0.73566	5.4530	-687.83
+ width	1	0.36503	5.8236	-686.75
+ wheel_base	1	0.18031	6.0083	-680.53
+ length	1	0.14103	6.0476	-679.24
+ drive_wheels	2	0.18333	6.0053	-678.63
+ city_mpg	1	0.07128	6.1173	-676.95
<none>			6.1886	-676.65
+ engine_size	1	0.03083	6.1578	-675.64
+ highway_mpg	1	0.01442	6.1742	-675.11
+ aspiration	1	0.01043	6.1782	-674.98
+ bore	1	0.00734	6.1813	-674.88
+ symboling	5	0.18572	6.0029	-672.71

Step: AIC=-697.22

log(price) ~ curb_weight + horsepower + body_style + num_cylinders

	Df	Sum of Sq	RSS	AIC
+ engine_location	1	0.44923	4.8051	-713.00
+ fuel_system	7	0.49578	4.7585	-702.94
+ width	1	0.18385	5.0705	-702.31
+ wheel_base	1	0.16011	5.0942	-701.38
+ drive_wheels	2	0.20420	5.0501	-701.11
+ length	1	0.14759	5.1067	-700.89
<none>			5.2543	-697.22
+ highway_mpg	1	0.02479	5.2295	-696.16
+ bore	1	0.02163	5.2327	-696.04
+ aspiration	1	0.00217	5.2521	-695.30

```

+ city_mpg          1    0.00092 5.2534 -695.25
+ engine_size       1    0.00033 5.2540 -695.23
+ symboling         5    0.09555 5.1588 -690.87

```

Step: AIC=-713

```

log(price) ~ curb_weight + horsepower + body_style + num_cylinders +
  engine_location

```

	Df	Sum of Sq	RSS	AIC
+ width	1	0.21505	4.5900	-720.11
+ wheel_base	1	0.19785	4.6072	-719.37
+ length	1	0.19028	4.6148	-719.04
+ fuel_system	7	0.44573	4.3593	-718.38
+ drive_wheels	2	0.19363	4.6114	-717.19
<none>			4.8051	-713.00
+ city_mpg	1	0.01683	4.7882	-711.70
+ bore	1	0.00256	4.8025	-711.11
+ aspiration	1	0.00199	4.8031	-711.09
+ engine_size	1	0.00173	4.8033	-711.07
+ highway_mpg	1	0.00098	4.8041	-711.04
+ symboling	5	0.04552	4.7596	-704.90

Step: AIC=-720.11

```

log(price) ~ curb_weight + horsepower + body_style + num_cylinders +
  engine_location + width

```

	Df	Sum of Sq	RSS	AIC
+ drive_wheels	2	0.22247	4.3676	-726.00
+ fuel_system	7	0.34347	4.2466	-721.59
+ wheel_base	1	0.07505	4.5150	-721.40
+ length	1	0.07313	4.5169	-721.31
<none>			4.5900	-720.11
+ city_mpg	1	0.02555	4.5645	-719.23
+ engine_size	1	0.00149	4.5885	-718.18
+ aspiration	1	0.00105	4.5890	-718.16
+ bore	1	0.00026	4.5898	-718.13
+ highway_mpg	1	0.00007	4.5899	-718.12
+ symboling	5	0.04874	4.5413	-712.24

Step: AIC=-726

```

log(price) ~ curb_weight + horsepower + body_style + num_cylinders +
  engine_location + width + drive_wheels

```

	Df	Sum of Sq	RSS	AIC
+ length	1	0.084116	4.2834	-727.87
<none>			4.3676	-726.00
+ city_mpg	1	0.034624	4.3329	-725.59

```

+ wheel_base    1  0.031842 4.3357 -725.46
+ bore          1  0.006558 4.3610 -724.30
+ fuel_system   7  0.261169 4.1064 -724.27
+ aspiration     1  0.000408 4.3671 -724.02
+ engine_size   1  0.000045 4.3675 -724.00
+ highway_mpg   1  0.000007 4.3675 -724.00
+ symboling     5  0.122401 4.2452 -721.66

```

Step: AIC=-727.87

```

log(price) ~ curb_weight + horsepower + body_style + num_cylinders +
  engine_location + width + drive_wheels + length

```

	Df	Sum of Sq	RSS	AIC
<none>			4.2834	-727.87
+ fuel_system	7	0.268983	4.0145	-726.78
+ aspiration	1	0.012685	4.2708	-726.46
+ bore	1	0.011565	4.2719	-726.41
+ city_mpg	1	0.010362	4.2731	-726.35
+ highway_mpg	1	0.005279	4.2782	-726.12
+ wheel_base	1	0.005165	4.2783	-726.11
+ engine_size	1	0.000095	4.2833	-725.88
+ symboling	5	0.126335	4.1571	-723.83

Backward Elimination Method

```

full_model <- refit_model_2
backward_model <- step(full_model, direction = "backward")

```

Start: AIC=-725.41

```

log(price) ~ (symboling + normalized_losses + make + fuel_type +
  aspiration + num_doors + body_style + drive_wheels + engine_location +
  wheel_base + length + width + height + curb_weight + engine_type +
  num_cylinders + engine_size + fuel_system + bore + stroke +
  compression_ratio + horsepower + peak_rpm + city_mpg + highway_mpg) -
  fuel_type - num_doors - normalized_losses - height - stroke -
  compression_ratio - peak_rpm - make - engine_type

```

	Df	Sum of Sq	RSS	AIC
- symboling	5	0.09602	3.7151	-730.20
- bore	1	0.00557	3.6247	-727.10
- wheel_base	1	0.01776	3.6369	-726.43
- engine_size	1	0.02207	3.6412	-726.20
- length	1	0.03150	3.6506	-725.68
- curb_weight	1	0.03301	3.6521	-725.60
<none>			3.6191	-725.41
- aspiration	1	0.03677	3.6559	-725.40
- width	1	0.05683	3.6759	-724.31
- fuel_system	7	0.31840	3.9375	-722.63
- horsepower	1	0.09274	3.7119	-722.37

- highway_mpg	1	0.18025	3.7994	-717.74
- drive_wheels	2	0.23740	3.8565	-716.77
- city_mpg	1	0.24504	3.8641	-714.37
- num_cylinders	6	0.45682	4.0759	-713.75
- engine_location	1	0.27613	3.8952	-712.78
- body_style	4	0.43259	4.0517	-710.94

Step: AIC=-730.2

log(price) ~ aspiration + body_style + drive_wheels + engine_location +
 wheel_base + length + width + curb_weight + num_cylinders +
 engine_size + fuel_system + bore + horsepower + city_mpg +
 highway_mpg

	Df	Sum of Sq	RSS	AIC
- bore	1	0.00599	3.7211	-731.88
- wheel_base	1	0.01238	3.7275	-731.54
- engine_size	1	0.02411	3.7392	-730.91
- aspiration	1	0.03501	3.7501	-730.33
<none>			3.7151	-730.20
- curb_weight	1	0.04022	3.7553	-730.05
- length	1	0.04068	3.7558	-730.03
- width	1	0.06747	3.7826	-728.62
- horsepower	1	0.09079	3.8059	-727.39
- fuel_system	7	0.34835	4.0635	-726.36
- highway_mpg	1	0.17171	3.8868	-723.21
- drive_wheels	2	0.22302	3.9381	-722.60
- num_cylinders	6	0.44042	4.1556	-719.90
- city_mpg	1	0.26956	3.9847	-718.26
- engine_location	1	0.34334	4.0585	-714.61
- body_style	4	0.46961	4.1847	-714.51

Step: AIC=-731.88

log(price) ~ aspiration + body_style + drive_wheels + engine_location +
 wheel_base + length + width + curb_weight + num_cylinders +
 engine_size + fuel_system + horsepower + city_mpg + highway_mpg

	Df	Sum of Sq	RSS	AIC
- wheel_base	1	0.01345	3.7346	-733.16
- engine_size	1	0.01812	3.7392	-732.91
- aspiration	1	0.03416	3.7553	-732.06
<none>			3.7211	-731.88
- length	1	0.04044	3.7616	-731.73
- curb_weight	1	0.04613	3.7673	-731.42
- width	1	0.06250	3.7836	-730.56
- horsepower	1	0.08819	3.8093	-729.22
- fuel_system	7	0.35657	4.0777	-727.67
- highway_mpg	1	0.17958	3.9007	-724.50

- drive_wheels	2	0.22148	3.9426	-724.37
- city_mpg	1	0.27036	3.9915	-719.92
- engine_location	1	0.33750	4.0586	-716.60
- body_style	4	0.46812	4.1892	-716.30
- num_cylinders	6	0.55368	4.2748	-716.27

Step: AIC=-733.16

log(price) ~ aspiration + body_style + drive_wheels + engine_location +
length + width + curb_weight + num_cylinders + engine_size +
fuel_system + horsepower + city_mpg + highway_mpg

	Df	Sum of Sq	RSS	AIC
- engine_size	1	0.02431	3.7589	-733.87
<none>			3.7346	-733.16
- aspiration	1	0.04293	3.7775	-732.88
- curb_weight	1	0.05379	3.7884	-732.31
- length	1	0.07069	3.8053	-731.43
- horsepower	1	0.07477	3.8093	-731.21
- width	1	0.08329	3.8179	-730.77
- fuel_system	7	0.36407	4.0986	-728.65
- highway_mpg	1	0.16685	3.9014	-726.46
- drive_wheels	2	0.24864	3.9832	-724.33
- city_mpg	1	0.25752	3.9921	-721.89
- body_style	4	0.45486	4.1894	-718.29
- engine_location	1	0.34862	4.0832	-717.40
- num_cylinders	6	0.55965	4.2942	-717.37

Step: AIC=-733.87

log(price) ~ aspiration + body_style + drive_wheels + engine_location +
length + width + curb_weight + num_cylinders + fuel_system +
horsepower + city_mpg + highway_mpg

	Df	Sum of Sq	RSS	AIC
- aspiration	1	0.02391	3.7828	-734.61
<none>			3.7589	-733.87
- length	1	0.06887	3.8277	-732.25
- width	1	0.08410	3.8430	-731.46
- fuel_system	7	0.34434	4.1032	-730.43
- curb_weight	1	0.11935	3.8782	-729.65
- horsepower	1	0.15493	3.9138	-727.83
- highway_mpg	1	0.16470	3.9236	-727.33
- drive_wheels	2	0.23479	3.9937	-725.81
- city_mpg	1	0.24934	4.0082	-723.09
- body_style	4	0.45551	4.2144	-719.11
- engine_location	1	0.33471	4.0936	-718.89
- num_cylinders	6	0.61776	4.3766	-715.59

Step: AIC=-734.61

```
log(price) ~ body_style + drive_wheels + engine_location + length +  
            width + curb_weight + num_cylinders + fuel_system + horsepower +  
            city_mpg + highway_mpg
```

	Df	Sum of Sq	RSS	AIC
<none>			3.7828	-734.61
- length	1	0.05795	3.8407	-733.58
- width	1	0.09678	3.8796	-731.58
- curb_weight	1	0.11569	3.8985	-730.61
- fuel_system	7	0.37251	4.1553	-729.92
- highway_mpg	1	0.14398	3.9268	-729.17
- drive_wheels	2	0.22693	4.0097	-727.01
- city_mpg	1	0.22935	4.0121	-724.89
- horsepower	1	0.29446	4.0772	-721.69
- engine_location	1	0.31365	4.0964	-720.75
- body_style	4	0.44994	4.2327	-720.24
- num_cylinders	6	0.59572	4.3785	-717.50

```
# VIF diagnostics  
vif(step_model)
```

	GVIF	Df	GVIF^(1/(2*Df))
curb_weight	14.045025	1	3.747669
horsepower	4.685355	1	2.164568
body_style	3.476962	4	1.168557
num_cylinders	6.126912	6	1.163064
engine_location	1.691866	1	1.300718
width	7.652570	1	2.766328
drive_wheels	2.915849	2	1.306746
length	9.000692	1	3.000115

```
vif(backward_model)
```

	GVIF	Df	GVIF^(1/(2*Df))
body_style	4.351092	4	1.201779
drive_wheels	3.836607	2	1.399545
engine_location	1.947158	1	1.395406
length	11.491981	1	3.389982
width	8.097027	1	2.845528
curb_weight	20.881382	1	4.569615
num_cylinders	41.470952	6	1.363993
fuel_system	51.792366	7	1.325707
horsepower	10.719238	1	3.274025
city_mpg	30.399394	1	5.513565
highway_mpg	27.931410	1	5.285018

```

# Refit backward model
backward_model <- lm(log(price) ~ body_style + drive_wheels + engine_location + length + width +
  ↪ curb_weight + num_cylinders + fuel_system + horsepower + highway_mpg, data = auto_clean)

# Decision Point 6 (Lei, Catherine, Jessica, 2026-03-30):
# We agreed to drop city_mpg variable from the backward model due to its strong correlation with
  ↪ highway_mpg, indicating multicollinearity. Since highway_mpg shows a stronger relationship with the
  ↪ response variable (price), it was retained while city_mpg was excluded to reduce redundancy.

# Observations:
# The model selected by stepwise regression and forward selection method is: log(price) ~ curb_weight +
  ↪ horsepower + body_style + num_cylinders + engine_location + width + drive_wheels + length.

# The model selected by backward elimination method is log(price) ~ body_style + drive_wheels +
  ↪ engine_location + length + width + curb_weight + num_cylinders + fuel_system + horsepower +
  ↪ highway_mpg.

```

Model Interaction Check

```

model_sf <- step_model
model_bw <- backward_model

# Main model for check the interaction of Stepwise and Forward model

model_sf_int1 <- update(model_sf, . ~ . + horsepower:drive_wheels)

model_sf_int2 <- update(model_sf, . ~ . + curb_weight:body_style)

model_sf_int3 <- update(model_sf, . ~ . + horsepower:body_style)

model_sf_int4 <- update(model_sf, . ~ . + curb_weight:horsepower)

model_bw_int1 <- update(model_bw, . ~ . + horsepower:drive_wheels)
model_bw_int2 <- update(model_bw, . ~ . + curb_weight:body_style)
model_bw_int3 <- update(model_bw, . ~ . + horsepower:highway_mpg)
model_bw_int4 <- update(model_bw, . ~ . + highway_mpg:body_style)

compare_models <- function(...) {
  models <- list(...)
  data.frame(
    Model = names(models),
    AIC = sapply(models, AIC),
    R2 = sapply(models, function(m) summary(m)$r.squared),
    Adjusted_R2 = sapply(models, function(m) summary(m)$adj.r.squared)
  )
}

compare_models(
  model_sf = model_sf,
  model_sf_int1 = model_sf_int1,
  model_sf_int2 = model_sf_int2,
  model_sf_int3 = model_sf_int3,
  model_sf_int4 = model_sf_int4
)

```

Model	AIC	R2	Adjusted_R2
-------	-----	----	-------------

```

model_sf          model_sf -161.1337 0.9149425 0.9069537
model_sf_int1 model_sf_int1 -157.3505 0.9150351 0.9060165
model_sf_int2 model_sf_int2 -159.6052 0.9176641 0.9078955
model_sf_int3 model_sf_int3 -157.7768 0.9169041 0.9070453
model_sf_int4 model_sf_int4 -159.5342 0.9151135 0.9066249

```

```

compare_models(
  model_bw = model_bw,
  model_bw_int1 = model_bw_int1,
  model_bw_int2 = model_bw_int2,
  model_bw_int3 = model_bw_int3,
  model_bw_int4 = model_bw_int4
)

```

	Model	AIC	R2	Adjusted_R2
model_bw	model_bw	-158.1546	0.9203298	0.9088168
model_bw_int1	model_bw_int1	-154.3771	0.9204188	0.9078534
model_bw_int2	model_bw_int2	-157.2410	0.9231170	0.9099240
model_bw_int3	model_bw_int3	-156.3754	0.9204182	0.9083884
model_bw_int4	model_bw_int4	-157.1743	0.9230912	0.9098938

```
anova(model_sf, model_sf_int1)
```

Analysis of Variance Table

Model 1: log(price) ~ curb_weight + horsepower + body_style + num_cylinders + engine_location + width + drive_wheels + length

Model 2: log(price) ~ curb_weight + horsepower + body_style + num_cylinders + engine_location + width + drive_wheels + length + horsepower:drive_wheels

	Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
1	181	4.2834				
2	179	4.2788	2	0.0046637	0.0976	0.9071

```
anova(model_sf, model_sf_int2)
```

Analysis of Variance Table

Model 1: log(price) ~ curb_weight + horsepower + body_style + num_cylinders + engine_location + width + drive_wheels + length

Model 2: log(price) ~ curb_weight + horsepower + body_style + num_cylinders + engine_location + width + drive_wheels + length + curb_weight:body_style

	Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
1	181	4.2834				
2	177	4.1464	4	0.13706	1.4627	0.2155

```
anova(model_sf, model_sf_int3)
```

Analysis of Variance Table

```
Model 1: log(price) ~ curb_weight + horsepower + body_style + num_cylinders +  
engine_location + width + drive_wheels + length
```

```
Model 2: log(price) ~ curb_weight + horsepower + body_style + num_cylinders +  
engine_location + width + drive_wheels + length + horsepower:body_style
```

	Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
1	181	4.2834				
2	177	4.1847	4	0.098785	1.0446	0.3857

```
anova(model_sf, model_sf_int4)
```

Analysis of Variance Table

```
Model 1: log(price) ~ curb_weight + horsepower + body_style + num_cylinders +  
engine_location + width + drive_wheels + length
```

```
Model 2: log(price) ~ curb_weight + horsepower + body_style + num_cylinders +  
engine_location + width + drive_wheels + length + curb_weight:horsepower
```

	Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
1	181	4.2834				
2	180	4.2748	1	0.0086121	0.3626	0.5478

```
anova(model_bw, model_bw_int1)
```

Analysis of Variance Table

```
Model 1: log(price) ~ body_style + drive_wheels + engine_location + length +  
width + curb_weight + num_cylinders + fuel_system + horsepower +  
highway_mpg
```

```
Model 2: log(price) ~ body_style + drive_wheels + engine_location + length +  
width + curb_weight + num_cylinders + fuel_system + horsepower +  
highway_mpg + drive_wheels:horsepower
```

	Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
1	173	4.0121				
2	171	4.0077	2	0.0044824	0.0956	0.9089

```
anova(model_bw, model_bw_int2)
```

Analysis of Variance Table

```
Model 1: log(price) ~ body_style + drive_wheels + engine_location + length +  
width + curb_weight + num_cylinders + fuel_system + horsepower +  
highway_mpg
```

```
Model 2: log(price) ~ body_style + drive_wheels + engine_location + length +  
width + curb_weight + num_cylinders + fuel_system + horsepower +  
highway_mpg + body_style:curb_weight
```

	Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
1	173	4.0121				
2	169	3.8718	4	0.14036	1.5316	0.1952

```
anova(model_bw, model_bw_int3)
```

Analysis of Variance Table

Model 1: $\log(\text{price}) \sim \text{body_style} + \text{drive_wheels} + \text{engine_location} + \text{length} + \text{width} + \text{curb_weight} + \text{num_cylinders} + \text{fuel_system} + \text{horsepower} + \text{highway_mpg}$

Model 2: $\log(\text{price}) \sim \text{body_style} + \text{drive_wheels} + \text{engine_location} + \text{length} + \text{width} + \text{curb_weight} + \text{num_cylinders} + \text{fuel_system} + \text{horsepower} + \text{highway_mpg} + \text{horsepower}:\text{highway_mpg}$

	Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
1	173	4.0121				
2	172	4.0077	1	0.0044493	0.191	0.6627

```
anova(model_bw, model_bw_int4)
```

Analysis of Variance Table

Model 1: $\log(\text{price}) \sim \text{body_style} + \text{drive_wheels} + \text{engine_location} + \text{length} + \text{width} + \text{curb_weight} + \text{num_cylinders} + \text{fuel_system} + \text{horsepower} + \text{highway_mpg}$

Model 2: $\log(\text{price}) \sim \text{body_style} + \text{drive_wheels} + \text{engine_location} + \text{length} + \text{width} + \text{curb_weight} + \text{num_cylinders} + \text{fuel_system} + \text{horsepower} + \text{highway_mpg} + \text{body_style}:\text{highway_mpg}$

	Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
1	173	4.0121				
2	169	3.8731	4	0.13906	1.517	0.1994

Observation:

None of the tested interaction terms produced a statistically significant improvement over the

↪ corresponding main-effects # model (all F-test p-values > 0.05), so we retained the simpler additive

↪ models without interactions.

Final Model Checking

These diagnostic tests are performed to evaluate the validity and adequacy of the candidate models.

The model selected by stepwise regression and forward selection method

```
# Generate regression diagnostic plots:
```

```
# Residuals vs Fitted
```

```
# Compute studentized residuals and fitted values for outlier detection
```

```
stud_res <- rstudent(step_model)
```

```
fitted_vals <- fitted(step_model)
```

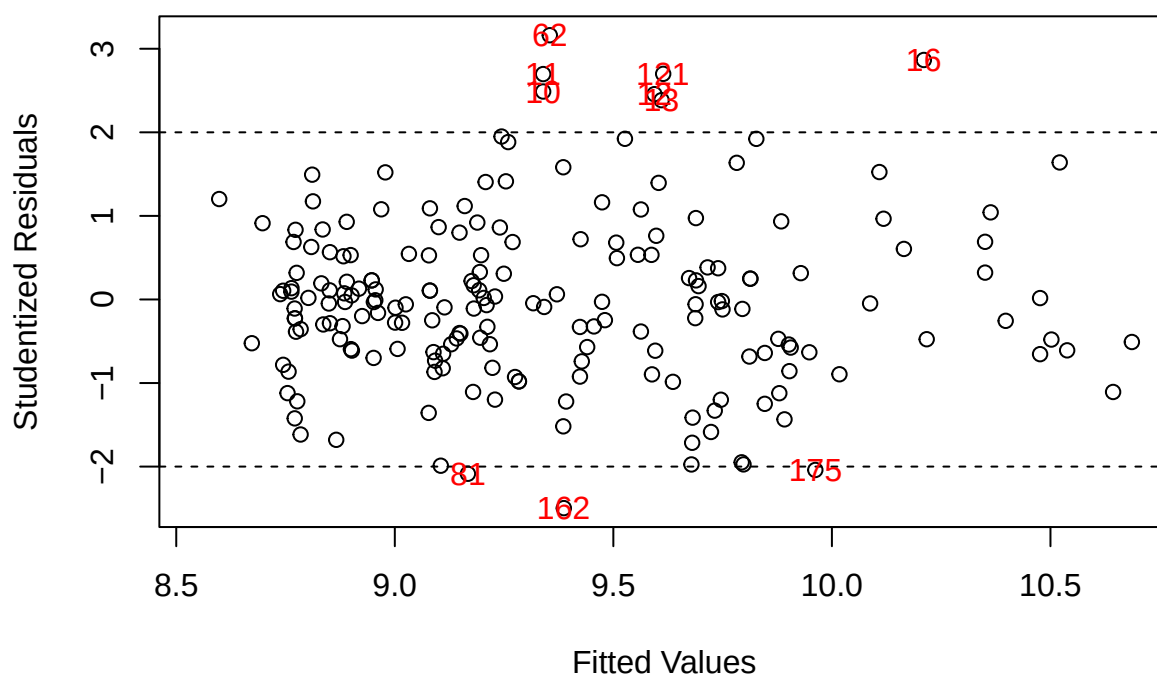
```
# Plot studentized residuals against fitted values to identify potential outliers
```

```
plot(fitted_vals, stud_res,  
     xlab = "Fitted Values",  
     ylab = "Studentized Residuals")
```

```
# Add reference lines at ±2 to highlight potential outliers
abline(h = c(-2, 2), lty = 2)

# Identify observations with large studentized residuals (|res| > 2)
outliers <- which(abs(stud_res) > 2)

# Label the detected outliers on the plot (with jitter to reduce overlap)
text(jitter(fitted_vals[outliers]),
     jitter(stud_res[outliers]),
     labels = outliers,
     col = "red")
```



```
# Print indices of potential outliers
which(abs(stud_res) > 2)
```

```
11 12 13 14 17 67 86 126 168 181
10 11 12 13 16 62 81 121 162 175
```

```
# Split variables into three groups for clearer table presentation
n <- ncol(auto_clean)
cols1 <- 1:floor(n/3)
cols2 <- (floor(n/3)+1):floor(2*n/3)
cols3 <- (floor(2*n/3)+1):n

# Display outlier observations across three tables to improve readability
kable(auto_clean[outliers, cols1], caption = "Outliers (Part 1)")
```

Table 9: Outliers (Part 1)

	symboling	normalized_losses	make	fuel_type	aspiration	num_doors	body_style	drive_wheels
11	2	192	bmw	gas	std	two	sedan	rwd

	symboling	normalized_losses	make	fuel_type	aspiration	num_doors	body_style	drive_wheels
12	0	192	bmw	gas	std	four	sedan	rwd
13	0	188	bmw	gas	std	two	sedan	rwd
14	0	188	bmw	gas	std	four	sedan	rwd
17	0	115	bmw	gas	std	two	sedan	rwd
67	0	115	mazda	diesel	std	four	sedan	rwd
86	1	125	mitsubishi	gas	std	four	sedan	fwd
126	3	186	porsche	gas	std	two	hatchback	rwd
168	2	134	toyota	gas	std	two	hardtop	rwd
181	-1	90	toyota	gas	std	four	sedan	rwd

```
kable(auto_clean[outliers, cols2], caption = "Outliers (Part 2)")
```

Table 10: Outliers (Part 2)

	engine_location	wheel_base	length	width	height	curb_weight	engine_type	num_cylinders	engine_size
11	front	101.2	176.8	64.8	54.3	2395	ohc	four	108
12	front	101.2	176.8	64.8	54.3	2395	ohc	four	108
13	front	101.2	176.8	64.8	54.3	2710	ohc	six	164
14	front	101.2	176.8	64.8	54.3	2765	ohc	six	164
17	front	103.5	193.8	67.9	53.7	3380	ohc	six	209
67	front	104.9	175.0	66.1	54.4	2700	ohc	four	134
86	front	96.3	172.4	65.4	51.6	2365	ohc	four	122
126	front	94.5	168.9	68.3	50.2	2778	ohc	four	151
168	front	98.4	176.2	65.6	52.0	2540	ohc	four	146
181	front	104.5	187.8	66.5	54.1	3131	dohc	six	171

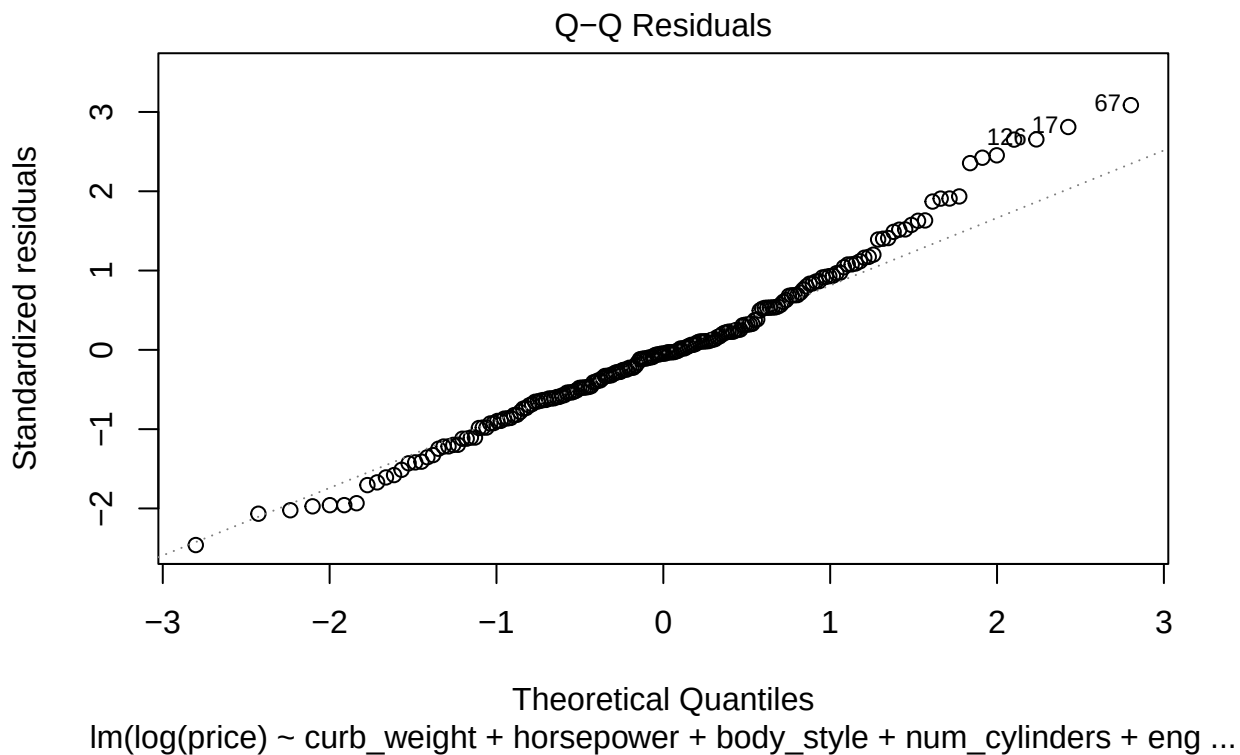
```
kable(auto_clean[outliers, cols3], caption = "Outliers (Part 3)")
```

Table 11: Outliers (Part 3)

	fuel_system	bore	stroke	compression_ratio	horsepower	peak_rpm	city_mpg	highway_mpg	price
11	mpfi	3.50	2.80	8.8	101	5800	23	29	16430
12	mpfi	3.50	2.80	8.8	101	5800	23	29	16925
13	mpfi	3.31	3.19	9.0	121	4250	21	28	20970
14	mpfi	3.31	3.19	9.0	121	4250	21	28	21105
17	mpfi	3.62	3.39	8.0	182	5400	16	22	41315
67	idi	3.43	3.64	22.0	72	4200	31	39	18344
86	2bbl	3.35	3.46	8.5	88	5000	25	32	6989
126	mpfi	3.94	3.11	9.5	143	5500	19	27	22018
168	mpfi	3.62	3.50	9.3	116	4800	24	30	8449
181	mpfi	3.27	3.35	9.2	156	5200	20	24	15690

```
# Normal Q-Q Plot
plot(step_model, which = 2)
```

Warning: not plotting observations with leverage one:
18, 46



```
# Cook's Distance Plot
cooks <- cooks.distance(step_model)
cooks_indexed <- data.frame(
  Obs = 1:length(cooks),
  Cook = cooks
)

vals <- paste(cooks_indexed$Obs, round(cooks_indexed$Cook, 4), sep = ": ")
vals <- c(vals, "")
cooks_mat <- matrix(vals, ncol = 5, byrow = TRUE)

kable(cooks_mat, caption = "Cook's Distance (Obs:Value)")
```

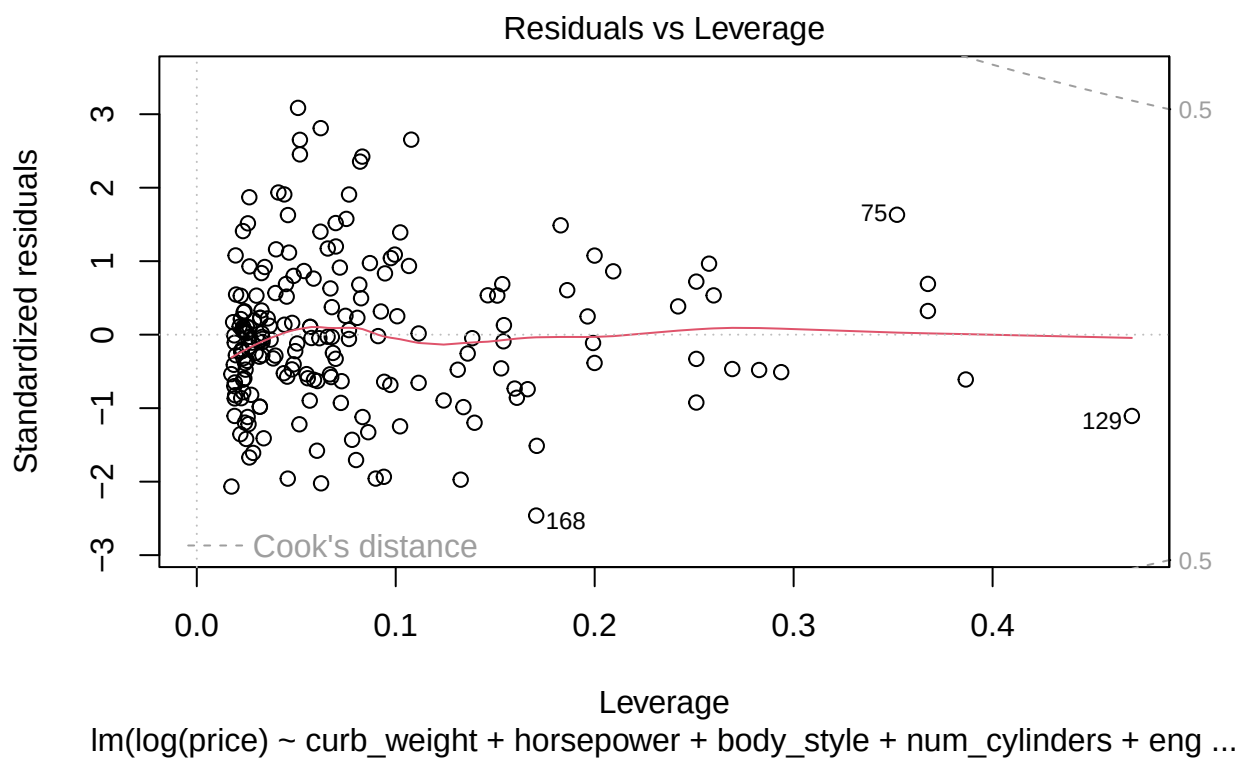
Table 12: Cook's Distance (Obs:Value)

1: 0.0021	2: 0.0161	3: 3e-04	4: 0.0053	5: 0.0026
6: 0.0027	7: 0.0078	8: 8e-04	9: 0	10: 0.0183
11: 0.0214	12: 0.0296	13: 0.0275	14: 0.0168	15: 0.0096
16: 0.0292	17: 0.0065	18: NaN	19: 0	20: 2e-04
21: 8e-04	22: 0	23: 1e-04	24: 1e-04	25: 2e-04
26: 7e-04	27: 0	28: 0	29: 0.006	30: 0.0036
31: 7e-04	32: 0	33: 0.0012	34: 1e-04	35: 0.004

36: 1e-04	37: 3e-04	38: 2e-04	39: 0.0073	40: 0.0026
41: 2e-04	42: 0.0331	43: 0	44: 0.003	45: 0
46: NaN	47: 0.0028	48: 2e-04	49: 1e-04	50: 5e-04
51: 0	52: 0.0159	53: 0.002	54: 0.0097	55: 0.0056
56: 5e-04	57: 0.002	58: 0.0018	59: 0	60: 0.0034
61: 0.0093	62: 0.0284	63: 0.0019	64: 0.0047	65: 0.018
66: 6e-04	67: 0.0051	68: 0.013	69: 0.006	70: 0.0803
71: 0.0028	72: 0.0022	73: 2e-04	74: 0	75: 0
76: 0	77: 1e-04	78: 3e-04	79: 0.0023	80: 0.0012
81: 0.0042	82: 0.0013	83: 0.0017	84: 0.0017	85: 0.0042
86: 0	87: 3e-04	88: 1e-04	89: 0.0016	90: 0
91: 0.0013	92: 1e-04	93: 0.0054	94: 0.0275	95: 0
96: 0	97: 0.0216	98: 0.0129	99: 0.0211	100: 0.0024
101: 0.0063	102: 0.0013	103: 0.0102	104: 0.0089	105: 0.0141
106: 0.0092	107: 1e-04	108: 0	109: 3e-04	110: 0
111: 1e-04	112: 6e-04	113: 6e-04	114: 9e-04	115: 1e-04
116: 1e-04	117: 5e-04	118: 0	119: 0	120: 0.0027
121: 0.0473	122: 0.0033	123: 0.0154	124: 0.0603	125: 9e-04
126: 0	127: 0	128: 2e-04	129: 0.0112	130: 0.0031
131: 0.0123	132: 0.005	133: 0.0042	134: 1e-04	135: 2e-04
136: 0.0023	137: 8e-04	138: 0	139: 0.0021	140: 1e-04
141: 0.0012	142: 0.0024	143: 0.0057	144: 0.0048	145: 0.0018
146: 0	147: 0	148: 0	149: 0.0028	150: 0.0045
151: 5e-04	152: 0	153: 0	154: 7e-04	155: 0
156: 0.0013	157: 0.0013	158: 0.0014	159: 0	160: 0.0037
161: 4e-04	162: 0.0692	163: 0.0262	164: 0.0045	165: 0.0061
166: 9e-04	167: 2e-04	168: 0.001	169: 5e-04	170: 1e-04
171: 1e-04	172: 0.0017	173: 0.0063	174: 0.0096	175: 0.0151
176: 0.0098	177: 1e-04	178: 7e-04	179: 0	180: 5e-04
181: 3e-04	182: 4e-04	183: 0	184: 0.0109	185: 0.0034
186: 0.0084	187: 0.0089	188: 0.0072	189: 0.0038	190: 0.0013
191: 0	192: 0.002	193: 0.0012	194: 4e-04	195: 0
196: 0.0017	197: 6e-04	198: 0.0058	199: 0.0071	

```
plot(step_model, which = 5)
```

Warning: not plotting observations with leverage one:
18, 46



```
# Observations with Cook's D > 0.5 or D > 1
which(cooks > 0.5)
```

```
named integer(0)
```

```
which(cooks > 1)
```

```
named integer(0)
```

```
# VIF diagnostics
vif(step_model)
```

	GVIF	Df	GVIF ^{1/(2*Df)}
curb_weight	14.045025	1	3.747669
horsepower	4.685355	1	2.164568
body_style	3.476962	4	1.168557
num_cylinders	6.126912	6	1.163064
engine_location	1.691866	1	1.300718
width	7.652570	1	2.766328
drive_wheels	2.915849	2	1.306746
length	9.000692	1	3.000115

```
# Observations:
```

```
# The three key assumptions (zero mean of residuals, constant variance, and normality) are generally
↳ satisfied, although slight deviations are observed in the tails. Based on Cook's distance, no
↳ influential observations are detected. All VIF values are below 5, indicating no serious
↳ multicollinearity concerns.
```

The model selected by stepwise regression and forward selection method

```
# Generate regression diagnostic plots:

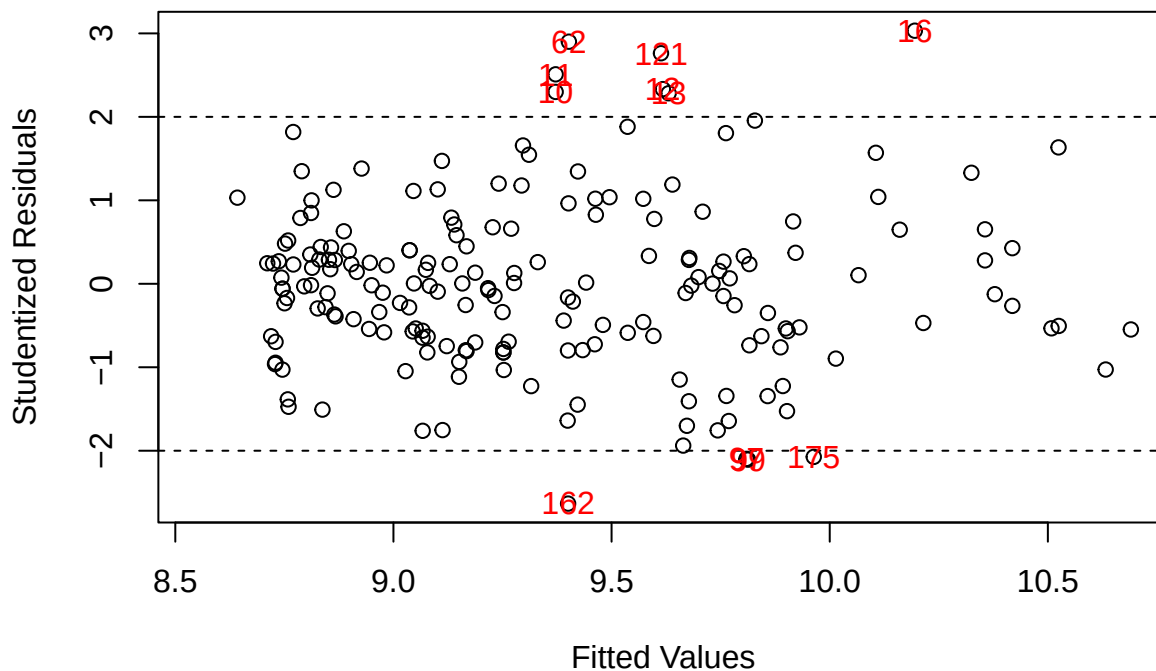
# Residuals vs Fitted
# Compute studentized residuals and fitted values for outlier detection
stud_res <- rstudent(backward_model)
fitted_vals <- fitted(backward_model)

# Plot studentized residuals against fitted values to identify potential outliers
plot(fitted_vals, stud_res,
     xlab = "Fitted Values",
     ylab = "Studentized Residuals")

# Add reference lines at  $\pm 2$  to highlight potential outliers
abline(h = c(-2, 2), lty = 2)

# Identify observations with large studentized residuals ( $|res| > 2$ )
outliers <- which(abs(stud_res) > 2)

# Label the detected outliers on the plot (with jitter to reduce overlap)
text(jitter(fitted_vals[outliers]),
     jitter(stud_res[outliers]),
     labels = outliers,
     col = "red")
```



```
# Print indices of potential outliers
which(abs(stud_res) > 2)
```

```
11 12 13 14 17 67 102 104 126 168 181
10 11 12 13 16 62 97 99 121 162 175
```

```
# Split variables into three groups for clearer table presentation
n <- ncol(auto_clean)
```

```
cols1 <- 1:floor(n/3)
cols2 <- (floor(n/3)+1):floor(2*n/3)
cols3 <- (floor(2*n/3)+1):n

# Display outlier observations across three tables to improve readability
kable(auto_clean[outliers, cols1], caption = "Outliers (Part 1)")
```

Table 13: Outliers (Part 1)

	symboling	normalized_losses	make	fuel_type	aspiration	num_doors	body_style	drive_wheels
11	2	192	bmw	gas	std	two	sedan	rwd
12	0	192	bmw	gas	std	four	sedan	rwd
13	0	188	bmw	gas	std	two	sedan	rwd
14	0	188	bmw	gas	std	four	sedan	rwd
17	0	115	bmw	gas	std	two	sedan	rwd
67	0	115	mazda	diesel	std	four	sedan	rwd
102	0	128	nissan	gas	std	four	sedan	fwd
104	0	108	nissan	gas	std	four	sedan	fwd
126	3	186	porsche	gas	std	two	hatchback	rwd
168	2	134	toyota	gas	std	two	hardtop	rwd
181	-1	90	toyota	gas	std	four	sedan	rwd

```
kable(auto_clean[outliers, cols2], caption = "Outliers (Part 2)")
```

Table 14: Outliers (Part 2)

	engine_location	wheel_base	length	width	height	curb_weight	engine_type	num_cylinders	engine_size
11	front	101.2	176.8	64.8	54.3	2395	ohc	four	108
12	front	101.2	176.8	64.8	54.3	2395	ohc	four	108
13	front	101.2	176.8	64.8	54.3	2710	ohc	six	164
14	front	101.2	176.8	64.8	54.3	2765	ohc	six	164
17	front	103.5	193.8	67.9	53.7	3380	ohc	six	209
67	front	104.9	175.0	66.1	54.4	2700	ohc	four	134
102	front	100.4	181.7	66.5	55.1	3095	ohcv	six	181
104	front	100.4	184.6	66.5	55.1	3060	ohcv	six	181
126	front	94.5	168.9	68.3	50.2	2778	ohc	four	151
168	front	98.4	176.2	65.6	52.0	2540	ohc	four	146
181	front	104.5	187.8	66.5	54.1	3131	dohc	six	171

```
kable(auto_clean[outliers, cols3], caption = "Outliers (Part 3)")
```

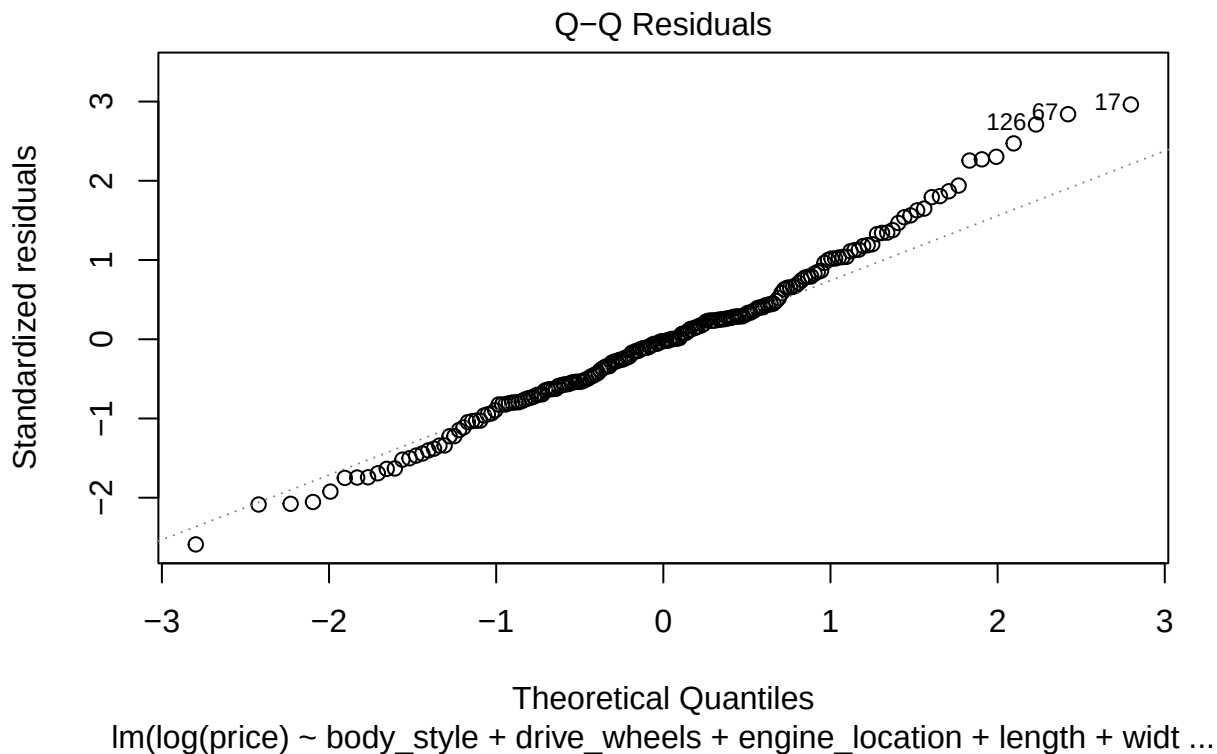
Table 15: Outliers (Part 3)

	fuel_system	bore	stroke	compression_ratio	horsepower	peak_rpm	city_mpg	highway_mpg	price
11	mpfi	3.50	2.80		8.8	101	5800	23	29 16430

	fuel_system	bore	stroke	compression_ratio	horsepower	peak_rpm	city_mpg	highway_mpg	price
12	mpfi	3.50	2.80	8.8	101	5800	23	29	16925
13	mpfi	3.31	3.19	9.0	121	4250	21	28	20970
14	mpfi	3.31	3.19	9.0	121	4250	21	28	21105
17	mpfi	3.62	3.39	8.0	182	5400	16	22	41315
67	idi	3.43	3.64	22.0	72	4200	31	39	18344
102	mpfi	3.43	3.27	9.0	152	5200	17	22	13499
104	mpfi	3.43	3.27	9.0	152	5200	19	25	13499
126	mpfi	3.94	3.11	9.5	143	5500	19	27	22018
168	mpfi	3.62	3.50	9.3	116	4800	24	30	8449
181	mpfi	3.27	3.35	9.2	156	5200	20	24	15690

```
# Normal Q-Q Plot
plot(backward_model, which = 2)
```

Warning: not plotting observations with leverage one:
18, 28, 43, 46, 55



```
# Cook's Distance Plot
cooks <- cooks.distance(backward_model)
cooks_indexed <- data.frame(
  Obs = 1:length(cooks),
  Cook = cooks
)

vals <- paste(cooks_indexed$Obs, round(cooks_indexed$Cook, 4), sep = ": ")
vals <- c(vals, "")
```

```
cooks_mat <- matrix(vals, ncol = 5, byrow = TRUE)

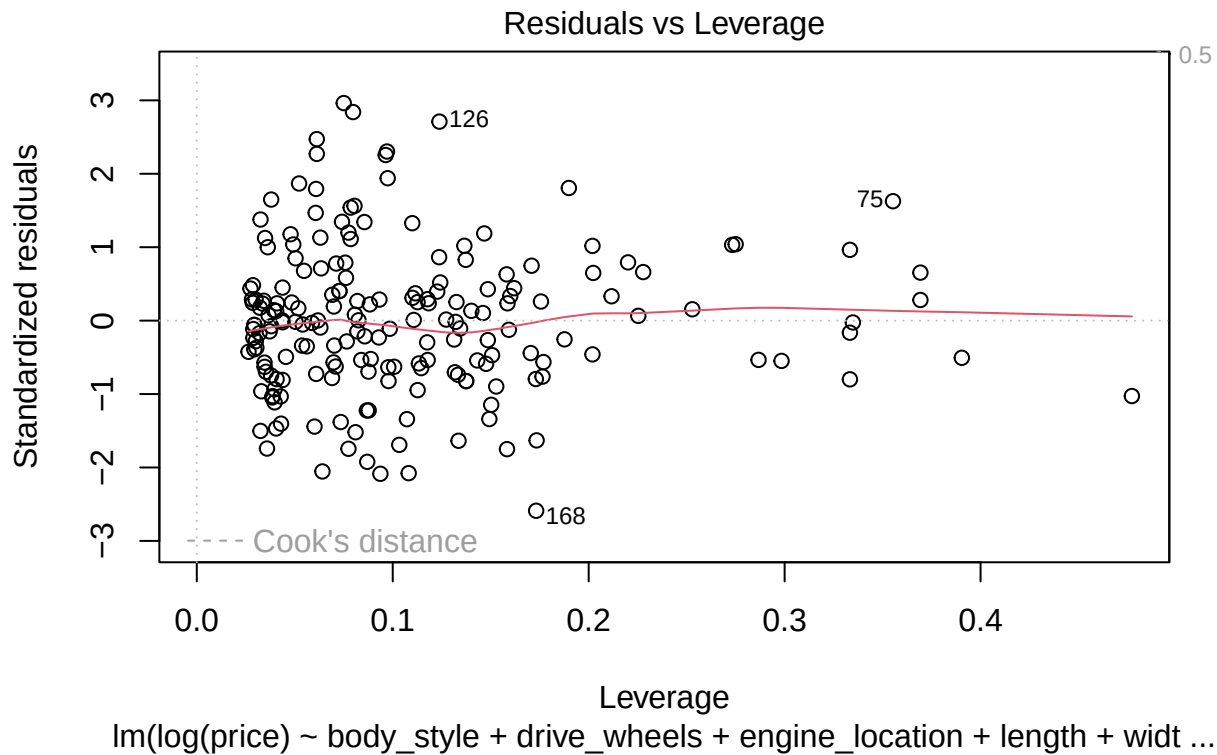
kable(cooks_mat, caption = "Cook's Distance (Obs:Value)")
```

Table 16: Cook's Distance (Obs:Value)

1: 0.0021	2: 0.0101	3: 0	4: 0.0041	5: 3e-04
6: 8e-04	7: 0.0048	8: 0.0011	9: 1e-04	10: 0.0129
11: 0.0153	12: 0.0219	13: 0.0209	14: 0.0156	15: 0.0082
16: 0.0274	17: 0.0084	18: NaN	19: 1e-04	20: 0
21: 5e-04	22: 1e-04	23: 2e-04	24: 0	25: 0
26: 0.0015	27: 5e-04	28: NaN	29: 0.0154	30: 0.0015
31: 0.0044	32: 4e-04	33: 4e-04	34: 0.0017	35: 0.0015
36: 0.0021	37: 3e-04	38: 0.0029	39: 0.0068	40: 0.0027
41: 0.0013	42: 0.0222	43: NaN	44: 5e-04	45: 0.0012
46: NaN	47: 0.0058	48: 1e-04	49: 3e-04	50: 2e-04
51: 1e-04	52: 0.0123	53: 5e-04	54: 0.0179	55: NaN
56: 0	57: 0.0011	58: 0.0033	59: 4e-04	60: 0.0054
61: 0.0074	62: 0.0269	63: 0.0015	64: 0.0041	65: 0.0158
66: 1e-04	67: 0.0044	68: 0.0063	69: 0.0049	70: 0.0561
71: 0.0032	72: 0.0016	73: 0	74: 1e-04	75: 0
76: 1e-04	77: 0	78: 0	79: 0.0063	80: 0.0042
81: 0.0043	82: 8e-04	83: 0.0042	84: 0.0042	85: 0.0029
86: 0.0019	87: 1e-04	88: 0	89: 0.002	90: 1e-04
91: 0.0014	92: 2e-04	93: 0.0056	94: 0.0294	95: 1e-04
96: 1e-04	97: 0.0201	98: 0.0121	99: 0.0173	100: 0.0017
101: 0.0056	102: 9e-04	103: 0.0136	104: 0.0098	105: 0.0127
106: 0.0159	107: 1e-04	108: 1e-04	109: 5e-04	110: 4e-04
111: 3e-04	112: 2e-04	113: 3e-04	114: 7e-04	115: 4e-04
116: 0	117: 2e-04	118: 0	119: 5e-04	120: 0.0023
121: 0.0399	122: 0.0018	123: 0.0096	124: 0.037	125: 0.0017
126: 3e-04	127: 2e-04	128: 4e-04	129: 0.0065	130: 0.0021
131: 0.0093	132: 0.0041	133: 0.0035	134: 4e-04	135: 4e-04
136: 0.0017	137: 4e-04	138: 0	139: 6e-04	140: 0.0015
141: 3e-04	142: 0.0011	143: 0.0026	144: 6e-04	145: 0.0012
146: 0	147: 1e-04	148: 1e-04	149: 0.0029	150: 0
151: 2e-04	152: 0	153: 0.0015	154: 1e-04	155: 8e-04
156: 0.0018	157: 0.0025	158: 3e-04	159: 2e-04	160: 0.0055
161: 0.0018	162: 0.054	163: 0.0215	164: 0.0051	165: 0.0051
166: 0.0013	167: 0	168: 0.0018	169: 0	170: 0
171: 0	172: 0.001	173: 0.0055	174: 0.0078	175: 0.0111
176: 0.0083	177: 0.0028	178: 0.002	179: 0.0017	180: 0.0014
181: 0.001	182: 0	183: 0	184: 0.005	185: 0.004
186: 0.0089	187: 0.0078	188: 0.0046	189: 0.0034	190: 0.0011
191: 0	192: 0.0018	193: 0.001	194: 3e-04	195: 0
196: 0.001	197: 7e-04	198: 0.0044	199: 0.008	

```
plot(backward_model, which = 5)
```

Warning: not plotting observations with leverage one:
18, 28, 43, 46, 55



```
# Observations with Cook's D > 0.5 or D > 1
which(cooks > 0.5)
```

```
named integer(0)
```

```
which(cooks > 1)
```

```
named integer(0)
```

```
# VIF diagnostics
vif(backward_model)
```

	GVIF	Df	GVIF ^{1/(2*Df)}
body_style	4.249107	4	1.198222
drive_wheels	3.715652	2	1.388381
engine_location	1.898974	1	1.378033
length	11.381909	1	3.373708
width	8.048364	1	2.836964
curb_weight	20.708043	1	4.550609
num_cylinders	36.358822	6	1.349121
fuel_system	43.746817	7	1.309817

horsepower	10.509144	1	3.241781
highway_mpg	7.944355	1	2.818573

```
# Observations:
# The three key assumptions (zero mean of residuals, constant variance, and normality) are generally
↪ satisfied, although slight deviations are observed in the tails. Based on Cook's distance, no
↪ influential observations are detected. All VIF values are below 5, indicating no serious
↪ multicollinearity concerns.
```

```
get_press <- function(model) {
  e <- resid(model)
  h <- hatvalues(model)
  sum((e / (1 - h))^2)
}

model_table <- data.frame(
  Model = c("Stepwise Model",
            "Backward Model"),

  n_predictors = c(length(coef(step_model)) - 1,
                    length(coef(backward_model)) - 1),

  R2 = c(summary(step_model)$r.squared,
          summary(backward_model)$r.squared),

  adj_R2 = c(summary(step_model)$adj.r.squared,
              summary(backward_model)$adj.r.squared),

  AIC = c(AIC(step_model), AIC(backward_model)),

  BIC = c(BIC(step_model), BIC(backward_model)),

  PRESS = c(get_press(step_model),
              get_press(backward_model))
)

# Round values
model_table$R2 <- round(model_table$R2, 4)
model_table$adj_R2 <- round(model_table$adj_R2, 4)
model_table$AIC <- round(model_table$AIC, 2)
model_table$BIC <- round(model_table$BIC, 2)

model_table
```

	Model	n_predictors	R2	adj_R2	AIC	BIC	PRESS
1	Stepwise Model	17	0.9149	0.9070	-161.13	-98.56	Inf
2	Backward Model	25	0.9203	0.9088	-158.15	-69.24	Inf

```
# Decision Point 7 (Lei, Catherine, Jessica, 2026-03-30):
# Based on the model comparison table, we agreed to select the stepwise model. The backward model does
↪ not show a meaningful improvement in adjusted R2, while the stepwise model has lower AIC and BIC,
↪ indicating a better trade-off between model fit and complexity.

# PRESS is not reliable in this case because several high-leverage points
# lead to infinite PRESS values. However, Cook's distance indicates that
# these observations are not influential, so no points are removed.
```

```
# The final model is: log(price) ~ curb_weight + horsepower + body_style + num_cylinders +  
↪ engine_location + width + drive_wheels + length.
```

Conclusions

Summary of Findings

The analysis aims to identify the key factors that influence car prices. The final model suggests that variables such as curb weight, horsepower, width, and engine location are among the most important predictors of price. In general, larger and more powerful cars tend to have higher prices, as indicated by the positive relationship between these variables and the response.

Categorical variables such as body style, number of cylinders, and drive wheels also play a role, capturing structural differences across vehicle types.

Interpretation of Effects

Most of the important predictors have a positive effect on price. For example, increases in curb weight, horsepower, and vehicle dimensions (such as width and length) are associated with higher prices. This indicates that size and performance are key drivers of vehicle value.

Limitations

Although three model selection methods are used to select the model, a different selection procedure (e.g. LASSO) may lead to a different final model, meaning the selected model may not be the same.

In addition, although multicollinearity is not a concern based on the VIF results, the analysis is based on observational data and may still suffer from omitted variable bias.

AI Disclosures

We used AI tools to assist with R code development. We used AI tools to help generate presentation slide template. We used AI tools for proofreading and improving clarity of writing.

References

- Cai, X., Lin, G., & Li, J. (2019). *Bayesian inverse regression for supervised dimension reduction with small datasets*.
- Samia, N. A., & Chakraborty, K. (2021). *Automobile price prediction using linear regression*.
- Schlimmer, J. (1985). *Automobile dataset*. UCI Machine Learning Repository. <https://archive.ics.uci.edu/dataset/10/automobile>
- Usman, M., & Pears, R. (2010). *Integration of data mining and data warehousing: A practical methodology*.